WOJSKOWA AKADEMIA TECHNICZNA

im. Jarosława Dąbrowskiego

WYDZIAŁ CYBERNETYKI



PRACA DYPLOMOWA

STUDIA I°

Temat pracy: **PROJEKT I IMPLEMENTACJA APLIKACJI DO ZARZĄDZANIA ZADANIAMI I FINANSAMI**

INFORMATYKA

(kierunek studiów)

INŻYNIERIA SYSTEMÓW

(specjalność)

Dyplomant:

Patrycja KLEIN

Promotor:

dr inż. Adrian WOŹNIAK

Warszawa 2025

OŚWIADCZENIE

~~*Wyrażam zgodę /*~~ *nie wyrażam zgody* [*]

na udostępnianie mojej pracy w czytelni Archiwum WAT.

Patrycja Klein

Dnia 30.05.2025 .....
(podpis)

[*]*Niepotrzebne skreślić*

Spis treści

Wstęp 4
Przegląd istniejących rozwiązań 5
Rozdział I 5
I.1. Aplikacja „Wallet” 5
I.2. Aplikacja „Spendee” 6
I.3. Aplikacja „Todoist” 7
Rozdział II. Projekt aplikacji 9
II.1. FURPS – wymagania funkcjonalne i niefunkcjonalne 9
II.2. Architektura aplikacji 12
II.3. Architektura kodu 13
II.4. Model logiczny danych 15
II.5. Diagram przypadków użycia 18
Rozdział III. Implementacja funkcjonalności 21
III.1. Narzędzia programistyczne i środowisko 21
III.2. Wykorzystane technologie 21
III.3. Wykorzystane biblioteki 23
III.4. Prezentacja aplikacji 25
III.5. Opis wybranych implementacji 43
III.6. Prezentacja modelu predykcji 49
Rozdział IV. Testy rozwiązania 55
Podsumowanie 58
Bibliografia 59
Spis rysunków 60
Załączniki 62

Wstęp

Współczesne życie codzienne wymaga efektywnego zarządzania czasem oraz finansami. Problemem wielu użytkowników pozostaje jednak brak skutecznych narzędzi integrujących oba te obszary w sposób prosty, przejrzysty i dostosowany do realnych potrzeb. Jak pokazał raport BIG InfoMonitor [1] z 2024 roku, aż 41% Polaków odczuwało stres związany z finansami. Różnice w poziomie dochodów oraz rozkładzie obowiązków sprawiają, że potrzeby w zakresie organizacji czasu i zarządzania finansami są zróżnicowane i często indywidualne. W efekcie, wzrasta potrzeba dostępu do rozwiązań, które łączą zarządzanie czasem i finansami w sposób elastyczny, intuicyjny i dostosowany do indywidualnych rytmów życia użytkowników.

Celem niniejszej pracy jest przedstawienie projektu aplikacji, która umożliwia jednoczesne monitorowanie wydatków, planowanie obowiązków oraz reagowanie na zmiany w dochodach, czy strukturze dnia.

W pierwszym rozdziale dokonano przeglądu wybranych aplikacji dostępnych na rynku, których funkcjonalności wpisują się w tematykę pracy. Przedstawiono ich główne cechy, wskazano mocne i słabe strony, a także zaprezentowano przykładowe interfejsy użytkownika pozwalające lepiej zrozumieć sposób działania poszczególnych rozwiązań.

W drugim rozdziale przedstawiono projekt systemu, wymagania funkcjonalne i niefunkcjonalne według modelu FURPS, logiczny model danych, architekturę aplikacji i kodu oraz diagram przypadków użycia, pozwalający zrozumieć podstawowe scenariusze działania systemu.

Rozdział trzeci opisuje proces tworzenia aplikacji od wyboru technologii po implementację głównych modułów, takich jak kalendarz, finanse, czy prognozy. Pokrótce omówiono ich działanie i zastosowane rozwiązania techniczne.

Ostatni rozdział poświęcono testowaniu aplikacji. Przeprowadzono testy manualne, sprawdzające działanie systemu w różnych scenariuszach, przeglądarkach i interakcjach. Skupiono się na poprawności danych, współdziałaniu modułów oraz stabilności działania.

Rozdział I. Przegląd istniejących rozwiązań

Współczesne życie codzienne stawia przed użytkownikami coraz większe wymagania związane z organizacją zarówno czasu, jak i budżetu. W odpowiedzi na te potrzeby powstają liczne aplikacje mobilne i webowe, które mają wspomagać planowanie, monitorowanie i analizowanie codziennych aktywności. Analiza aplikacji „Wallet", „Spendee" oraz „Todoist" pozwala zrozumieć, w jaki sposób różne narzędzia realizują funkcje związane z zarządzaniem finansami oraz organizacją czasu. Wspomniane narzędzia zostały poddane analizie pod kątem dostępnych funkcji, przejrzystości interfejsu użytkownika oraz możliwości personalizacji.

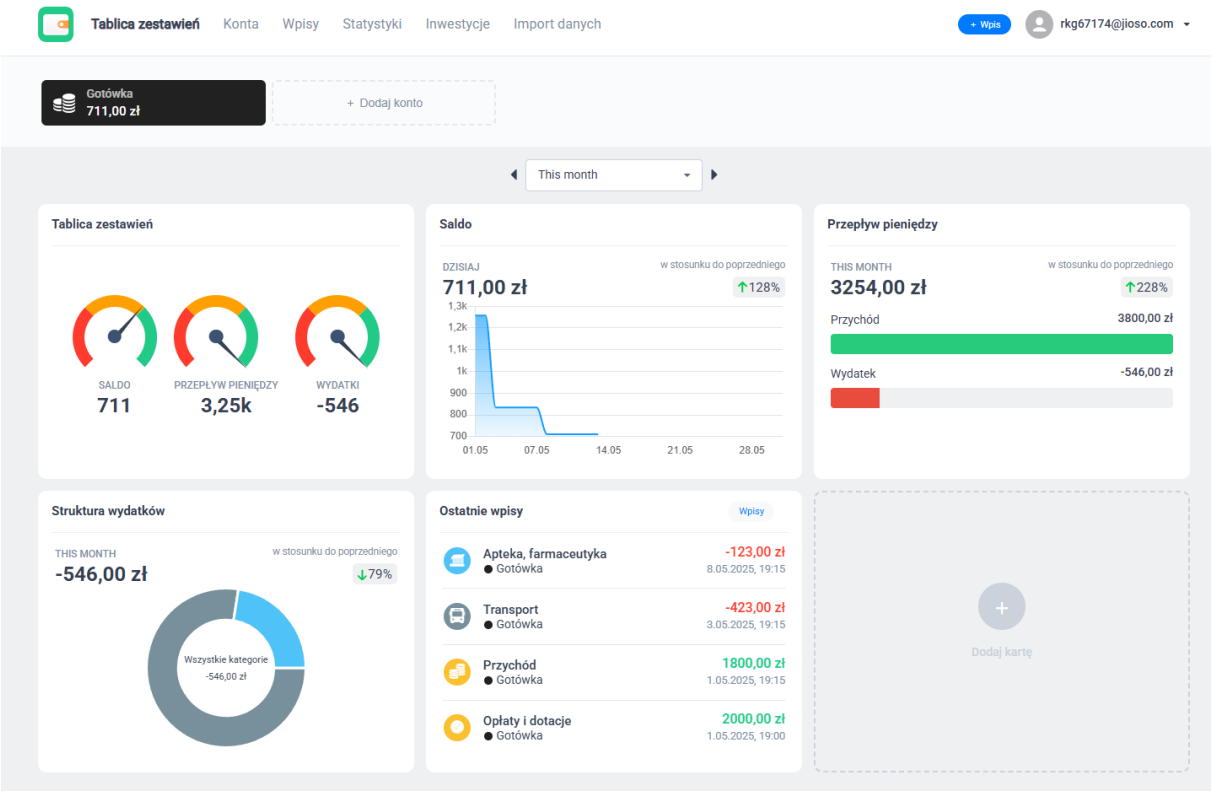
I.1. Aplikacja „Wallet"

Pierwszym z analizowanych rozwiązań jest aplikacja o nazwie „Wallet" [2] firmy BudgetBakers, dostępna zarówno w wersji webowej oraz w formie aplikacji mobilnej. Strona ta umożliwia kontrolowanie budżetu domowego poprzez monitorowanie transakcji. W rozważanej wersji darmowej subskrypcji dostępne są podstawowe funkcje, takie jak ręczne dodawanie operacji przychodów, wydatków oraz transferów między kontami. Istnieje również możliwość dodania do trzech kont rachunków. Strona w prosty i przejrzysty sposób pozwala użytkownikowi na dodanie wpisów z możliwością wyboru takich rzeczy jak: daty, kwoty oraz kategorie. Użytkownik ma do dyspozycji także wykresy pokazujące kluczowe parametry finansowe takie jak: miernik salda, przepływy przychodów i wydatków oraz wykres liniowy zmian salda na przestrzeni czasu (Rys.1). Dzięki możliwości kategoryzowania wydatków, użytkownik widzi strukturę kosztów w układzie kołowym. Wykresy w przystępny sposób dla użytkownika pokazują sytuację finansową.

Jedną z kolejnych zalet aplikacji są zakładki „Konta" i „Wpisy", które dają widok na wprowadzone przez nas informacje. Istnieje możliwość zobaczenia wykresów liniowych dla poszczególnych kont pokazujących jak saldo zmieniało się na przestrzeni dni na danym koncie. Dostępne są również zestawienia przychodów i wydatków z możliwością prezentowania ich w przystępny sposób dla użytkownika poprzez możliwości filtrowania transakcji za pomocą wyboru kategorii i zakresu czasu. Kluczową funkcją wersji webowej są raporty w zakładce „Statystyki", które mają możliwość wybrania zakresu czasu do zestawienia ze sobą paru kategorii lub kont razem. Rozważana funkcja pozwala zestawić dane z aktualnego okresu z danymi z okresu wcześniejszego.

Strona nie posiada możliwości zaplanowania wydatków z wyprzedzeniem, jak również wprowadzenia opłat cyklicznych. W wersji darmowej nie ma możliwości integracji kont z bankami. Brakuje również funkcji wyznaczania celów finansowych, co uniemożliwia orientacyjną kontrolę nad oszczędnościami.

Dzięki tym rozwiązaniom, aplikacja w wersji darmowej jest intuicyjna w obsłudze, wystarczająca do notowania codziennych operacji oraz szybkiego wglądu w podstawowe wskaźniki finansowe. Standardowy użytkownik niepotrzebujący rozbudowanych analiz, otrzymuje najważniejsze funkcjonalności bez dodatkowych opłat. Dla wymagającego użytkownika aplikacja „Wallet" może nie być wystarczająca z uwagi na ograniczony zakres funkcjonalności.



Rys. 1. Widok aplikacji Wallet

Źródło: Zrzut ekranu z strony web.budgetbakers.com. [opracowanie własne]

I.2. Aplikacja „Spendee"

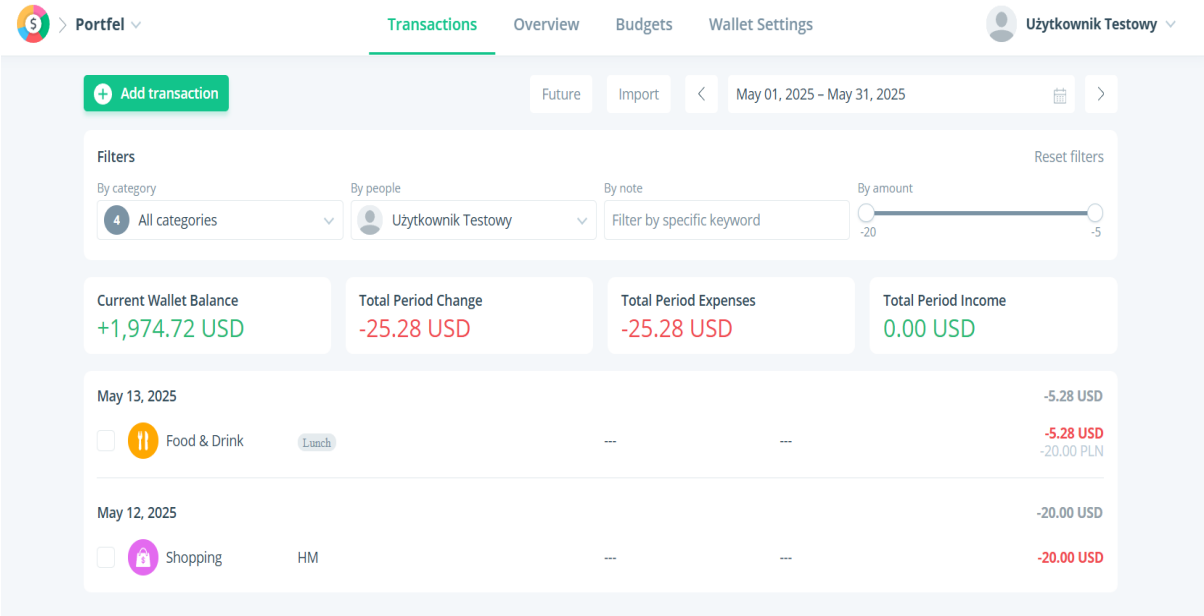
Drugim analizowanym narzędziem jest aplikacja „Spendee" [3], która również jest dostępna zarówno w wersji webowej, jak i mobilnej. W bezpłatnym planie pozwala na ręczne wprowadzanie osobistych operacji finansowych. Użytkownik może posiadać tylko jeden portfel i w jego obrębie rejestrować operacje przychodów, wydatków oraz korekty transakcji. Formularz dodawania transakcji jest intuicyjny, posiada podstawowe funkcjonalności, takie jak: możliwość dodania kwoty, daty, kategorii, opcjonalnie użytkownik może dodać również notatkę, osobę przypisaną do transakcji oraz walutę. System automatycznie przelicza kwoty na walutę portfela, co ułatwia śledzenie wydatków zagranicznych.

Widok zakładki „Transactions" prezentuje bieżące informacje o saldzie portfela wraz z jego transakcjami, które są uporządkowane chronologicznie (Rys.2). Wbudowane filtry pozwalają na szybkie odnalezienie interesującego wpisu. Dostępna

jest zakładka „Overview” oferująca podstawową analitykę wizualną, taką jak: wykresy salda w czasie, histogram zmian dziennych oraz pierścieniowy diagram struktury wydatków według kategorii. Widoki mają możliwość przełączania między dniami, tygodniami i miesiącami, dzięki czemu użytkownik może dostosować analizę do interesującego go przedziału czasu.

Istotną cechą rozwiązania jest możliwość zdefiniowania budżetu powiązanego z wybranymi kategoriami. Daje to możliwość pilnowania finansów, aby w wyznaczonym okresie nie przekraczały ustalonej sumy w wybranych kategoriach. Aplikacja wyświetla procent wykorzystanego limitu, pozostałą kwotę do wykorzystania, sugerowaną kwotę dzienną na wydatki, co pomaga kontrolować wydatki bez konieczności ręcznych obliczeń. W wersji darmowej użytkownicy są ograniczeni do możliwości tworzenia jednego budżetu jednocześnie, co ogranicza możliwość rozplanowania budżetu szczegółowo na pojedyncze kategorie.

Do pobocznych przydatnych funkcji może też służyć możliwość importowania transakcji za pomocą pliku CSV, czy możliwość eksportowania danych do arkusza kalkulacyjnego. Aplikacja nie ma możliwości synchronizacji z bankami, nie oferuje również możliwości ustawiania celów finansowych, wydatków stałych, ani prognoz oszczędnościowych. Użytkujący aplikację może napotkać szybko bariery związane z brakiem możliwości pełnej kontroli nad swoimi finansami, lecz dla standardowego użytkownika powinna w zupełności być wystarczająca.



Rys. 2. Widok aplikacji Spendee

Źródło: Zrzut ekranu z aplikacji app.spendee.com. [opracowanie własne]

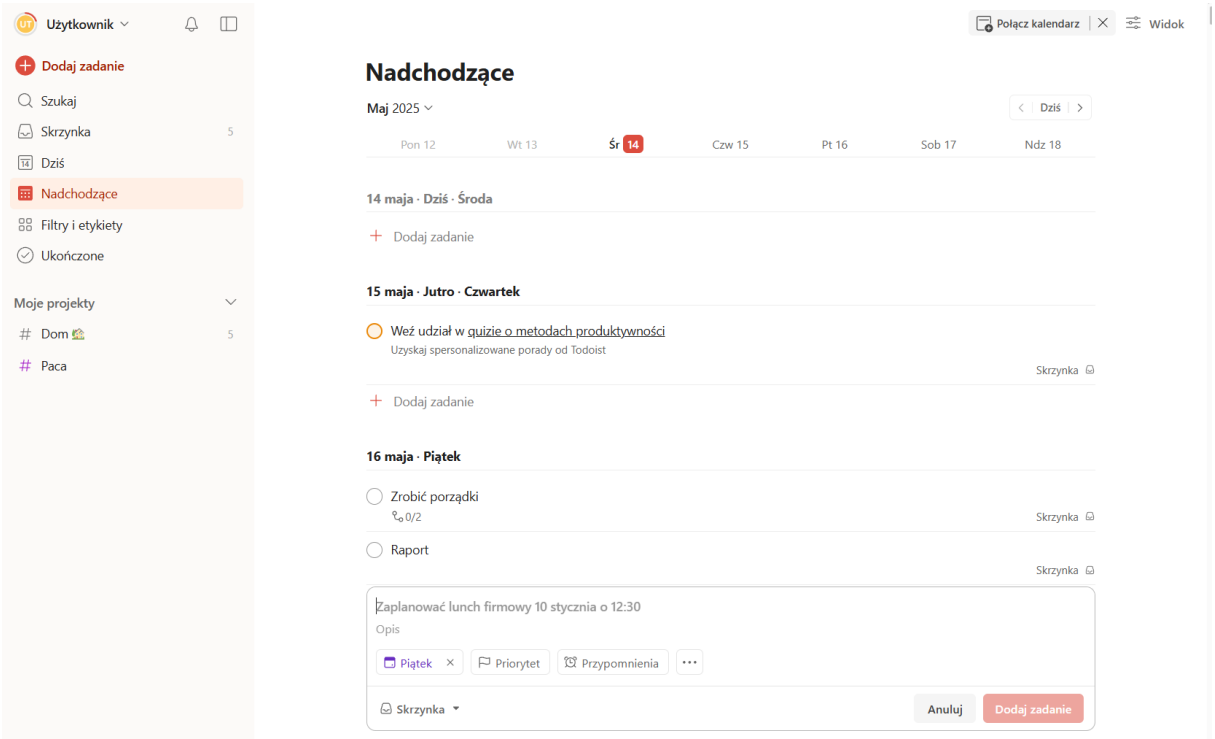
I.3. Aplikacja „Todoist”

Ostatnim analizowanym rozwiązaniem jest aplikacja „Todoist” [4] od firmy Doist, dostępna zarówno w przeglądarce, jak i wersji mobilnej. Dostępna jest zarówno w planie darmowym, jak i płatnym. W dalszej części rozważana będzie wersja bezpłatna, która

pozwala na planowanie oraz bieżące monitorowanie zadań. Użytkownik otrzymuje czytelny podział na widoki: „Dziś”, „Nadchodzące” oraz własne projekty, do których może przypisywać zadania i podzadania (Rys.3). Zaletą aplikacji jest obsługa podzadań, priorytetów oraz etykiet, co pozwala na grupowanie czynności według kontekstu lub priorytetu. Aplikacja posiada również podstronę „Ukończone”, która rejestruje historię zadań wykonanych przez użytkownika. Dodatkowo umożliwia monitorowanie produktywności poprzez wykres prezentujący liczbę zadań ukończonych każdego dnia.

Dodatkową funkcją planowania jest opcja jednostronnej integracji z „Google Calendar” od firmy Google. Wydarzenia wprowadzone przez użytkownika „Todoist” automatycznie pojawiają się w kalendarzu, co ułatwia szersze planowanie czasu. Aplikacja posiada również możliwość eksportu projektów do pliku CSV, co jest przydatne przy archiwizacji lub analizie w arkuszu kalkulacyjnym.

Ograniczeniem darmowej wersji jest posiadanie maksymalnie 5 projektów naraz, co przy większych przedsięwzięciach może nie spełniać swojej roli. Brakuje również możliwości ustawienia terminu wykonania zadania oraz automatycznych przypomnień mówiących o tym, że jakieś zadanie trzeba wykonać w najbliższym czasie. Niestety aplikacja sama nie posiada w sobie kalendarza z możliwością wprowadzania wydarzeń. Dla niewymagającego użytkownika aplikacja jest intuicyjnym i wydajnym narzędziem do codziennego planowania zadań. Ograniczenia liczby projektów, brak automatycznych przypomnień oraz okrojone opcje współpracy mogą jednak stanowić barierę dla użytkowników zarządzających wieloma równoległymi inicjatywami.



Rys. 3. Widok aplikacji Todoist

Źródło: Zrzut ekranu z aplikacji app.todoist.com [opracowanie własne]

Rozdział II. Projekt aplikacji

Ten rozdział poświęcony jest opisowi projektu aplikacji, z uwzględnieniem sprawdzonych metod i koncepcji stosowanych w inżynierii oprogramowania. Na początku omówiono metodyka FURPS, dzięki której uporządkowano wymagania funkcjonalne i niefunkcjonalne. Kolejnym punktem rozdziału jest logiczny model baz danych, czyli reprezentacja graficzna schematu bazy wykorzystanego przy implementacji. W dalszej części jest zaprezentowana architektura aplikacji i jej kodu, z wyszczególnieniem jej głównych warstw oraz powiązań między nimi. Przedstawiony również został diagram przypadków użycia, który ilustruje podstawowe funkcjonalności systemu z perspektywy użytkownika.

II.1. FURPS – wymagania funkcjonalne i niefunkcjonalne

Klasyfikacja FURPS (Functionality, Usability, Reliability, Performance, Supportability) [5] pozwala klasyfikować wymagania stawiane systemom informatycznym. Pozwala na określenie wymagań podczas etapu projektu:

- Functionality – funkcjonalność, określa jakie operacje i usługi musi zapewnić oprogramowanie.
- Usability – użyteczność, odnosi się do wizualnych aspektów, łatwości w obsłudze obejmujących czytelność interfejsu i estetykę.
- Reliability – jest to niezawodność, określa zdolność systemu do stabilnego działania, odporność na błędy, zapewnienie poprawnego czasu działania bez awarii.
- Performance – wydajność aplikacji, dotyczy szybkości działania jak i również efektywnego wykorzystania zasobów.
- Supportability – łatwość utrzymania, diagnozowania, testowania oraz adaptacji systemu w różnych środowiskach i warunkach eksploatacyjnych.

Przy użyciu tej klasyfikacji zdefiniowano wymagania funkcjonalne oraz niefunkcjonalne w tabelach:

- Tabela 1 przedstawia główne wymagania niefunkcjonalne, które odnoszą się do jakości działania systemu oraz jego pożądanych właściwości (Tab. 1).
- Tabela 2 zawiera wymagania funkcjonalne, określające konkretne działania i funkcje, jakie system powinien realizować w ramach swojego przeznaczenia (Tab. 2).

Tab. 1. Tabela wymagań niefunkcjonalnych projektowanego systemu

Wymagania	Kategoria FURPS
Interfejs graficzny pozwala użytkownikowi dotrzeć do dowolnej funkcji w maksymalnie 3 kliknięcia	Użyteczność
Formularze walidują dane na bieżąco, pokazując błędy w czytelny sposób	Użyteczność
Wykresy i podsumowania są prezentowane w graficznej, zrozumiałej formie	Użyteczność
System zapewnia integralność i spójność danych dla użytkownika	Niezawodność
System nie dopuszcza do utraty danych podczas zamknięcia przeglądarki	Niezawodność
System synchronizuje dane w mniej niż 1 sekundę	Niezawodność
Hasła użytkowników są przechowywane w postaci zaszyfrowanej, nigdy w postaci jawnej	Niezawodność
Aplikacja działa płynnie podczas równoczesnego wyświetlania wykresów i danych	Wydajność
Filtrowanie i sortowanie list odbywa się bez konieczności odświeżania strony	Wydajność
Czas reakcji systemu na operację nie przekracza 1 sekundy	Wydajność
System zapewnia poprawne działanie i wyświetlanie interfejsu dla przeglądarek Google Chrome (od wersji 79.0.3945.130), Mozilla Firefox (od wersji 72.0.1) oraz Opera (od wersji 66.0.3515.36.)	Wsparcie
System dział poprawnie dla większości komputerów z systemem operacyjnym Windows (Windows 7, Windows 8, Windows 8.1, Windows 10), bez potrzeby instalowania dodatkowego oprogramowania poza przeglądarką	Wsparcie

Tab. 2. Tabela wymagań funkcjonalnych projektowanego systemu

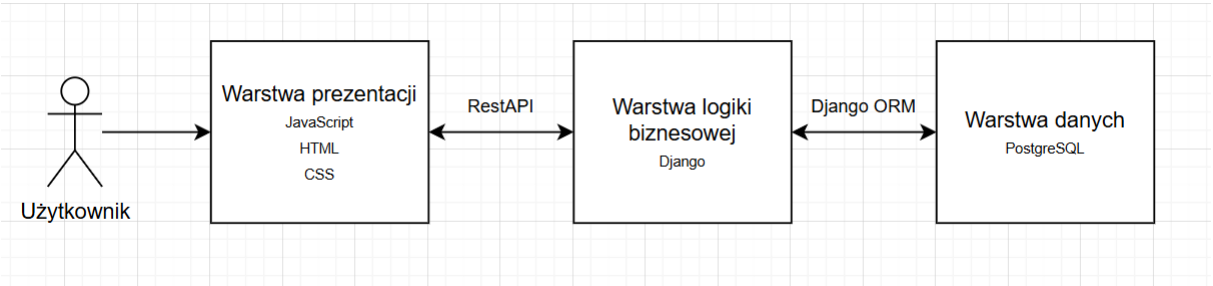
Wymagania funkcjonalne
Możliwość rejestracji i logowania
Możliwość wylogowania
Tworzenie, edytowanie, usuwanie zadań krótkoterminowych oraz śledzenie ich stanu
Tworzenie, edytowanie, usuwanie zadań długoterminowych oraz śledzenie ich stanu

Możliwość przypisania podzadania do zadania długoterminowego
Tworzenie, edytowanie, usuwanie podzadań oraz śledzenie ich postęp
Możliwość oznaczania zadań długoterminowych jako ulubione
Wyświetlanie informacji i miernika postępu dla zadania długoterminowego
Tworzenie, edytowanie, usuwanie wydarzeń jednorazowych i cyklicznych oraz śledzenie ich stanu
Wyświetlanie kalendarza w widoku dziennym, tygodniowym i miesięcznym
Wyświetlanie wydarzenia w odpowiednich widokach kalendarza
Tworzenie, edytowanie, usuwanie nawyków oraz śledzenie ich stanu
Wyświetlanie wykresu przebiegu realizacji nawyków
Wyświetlanie statystyk dla nawyków
Wyświetlanie statystyk finansowych dla użytkownika
Tworzenie, edytowanie, usuwanie kont finansowych oraz śledzenie ich stanu
Dodawanie, usuwanie transakcji dla przychodów, wydatków i transferów między kontami finansowymi
Wyświetlanie historii transakcji finansowych
Przeglądanie, filtrowanie i sortowanie transakcji finansowych po kryteriach
System zlicza sumę transakcji po kryteriach
Tworzenie, edytowanie, usuwanie opłat stałych, zobowiązań i celów finansowych oraz śledzenie ich stanu
Realizowanie wpłaty środków na cele, spłatę rachunku, spłatę zobowiązań
Obliczanie i wyświetlanie sugerowanej kwoty do odłożenia na każdy cel finansowy i opłatę roczną
Wyświetlanie postępów realizacji zobowiązań i celów finansowych
Obliczanie i wyświetlanie ratę do opłacenia dla zobowiązań cyklicznych
Wyświetlanie i monitorowanie miesięcznego budżetu
Wyświetlanie realizacji miesięcznego budżetu

Wyświetlanie wydatków z podziałem na kategorie na wykresie kołowym
Generowanie prognozy przyszłych przychodów oraz wydatków na podstawie dotychczasowych danych użytkownika
Wyświetlanie wyników prognozy i danych historycznych uwzględniających przychody i wydatki

II.2. Architektura aplikacji

Aplikacja została zaprojektowana zgodnie z trójwarstwową architekturą, zapewniającą jasny podział obowiązków pomiędzy poszczególnymi komponentami (Rys. 4). Architektura ta składa się z następujących warstw:



Rys. 4. Schemat komunikacji aplikacji

Źródło: Opracowanie własne.

- Warstwa prezentacji (Frontend) – odpowiedzialna za interfejs użytkownika, który został zbudowany przy użyciu technologii HTML, CSS oraz JavaScript. Odpowiada za to, co widzi użytkownik.
- Warstwa logiki biznesowej (Backend) – zrealizowana w technologii Django oraz Django REST Framework. Warstwa ta obejmuje komponenty takie jak widoki aplikacji, serializery danych oraz zestaw reguł biznesowych obsługujących logikę aplikacji. API aplikacji pozwala na bezpieczną oraz efektywną komunikację pomiędzy warstwą prezentacji a warstwą danych, realizując wszystkie kluczowe funkcje biznesowe systemu.
- Warstwa danych (Database) - została zrealizowana z wykorzystaniem modeli Django, które mapują się na relacyjne tabele bazy danych PostgreSQL. Dzięki mechanizmowi ORM (Object-Relational Mapping) aplikacja umożliwia wygodne operacje na danych bez konieczności pisania zapytań SQL. Dane przekazywane są pomiędzy warstwą backendową a frontem za pośrednictwem API REST.

Zastosowanie trójwarstwowej architektury umożliwia niezależny rozwój każdej z warstw, ułatwia utrzymanie oraz modyfikację poszczególnych funkcjonalności, a

także zwiększa ogólne bezpieczeństwo danych oraz wydajność aplikacji poprzez możliwość selektywnego skalowania poszczególnych komponentów.

II.3. Architektura kodu

Architektura kodu aplikacji została zaprojektowana zgodnie z aktualnymi standardami wykorzystywanymi w aplikacjach webowych. W projekcie zastosowano wyraźny podział na warstwę prezentacji, logiki biznesowej oraz danych, co sprzyja przejrzystości rozwiązania i ułatwia jego rozwój. Każda z tych warstw pełni odrębną funkcję – backend realizuje logikę i obsługuje dane przez interfejsy API, natomiast cała obsługa interfejsu użytkownika oraz prezentacja danych realizowana jest po stronie klienta w przeglądarce.

II.3.1. Architektura kodu warstwy prezentacji

Architektura kodu warstwy prezentacji w aplikacji została oparta na łączeniu statycznych plików HTML z dynamicznymi komponentami JavaScript. Rozszerzanie interfejsu lub dodawanie nowych funkcjonalności sprowadza się najczęściej do edycji odpowiednich plików i powiązanych z nimi modułów JavaScript.

- Pliki HTML – Wykorzystywane są głównie do ładowania podstawowej struktury poszczególnych sekcji aplikacji. Nie generują dynamicznych danych po stronie serwera – za to odpowiadają za ogólny układ strony i zawierają elementy wspólne, takie jak menu główne. Wszystkie dane wyświetlane użytkownikowi są pobierane i wstawiane do tych struktur dynamicznie, już po stronie przeglądarki, przez JavaScript.
- Pliki statyczne JavaScript – Każdy sekcja aplikacji posiada własny plik JS odpowiedzialny za interaktywność. Skrypty reagują na akcje użytkownika, które wykonuje na stronie, takie jak wypełnianie formularzy czy kliknięcie w przycisk. Komunikacja z backendem występuje przy użyciu zapytań AJAX (Asynchronous JavaScript and XML), czyli asynchroniczne przesyłanie danych pomiędzy przeglądarką a serwerem. Nie wymaga to przeładowania strony, użytkownik po dodaniu nowych danych do strony, widzi natychmiastową aktualizację interfejsu. Użytkownik otrzymuje płynność i wygodę korzystania z aplikacji.

- Pliki statyczne CSS – Odpowiadają za wizualną część aplikacji, dbając o estetykę całości. Zewnętrzne arkusze dają nam możliwość na łatwą zmianę kolorów, rozmiaru elementów i ich układu. Dzięki możliwości wykorzystania reguł, wszystkie strony aplikacji dostosowują automatycznie wygląd do rozdzielczości ekranu. Każda strona aplikacji, posiada swój osobny plik CSS.

Tak zorganizowana warstwa prezentacji pozwala na łatwą rozbudowę interfejsu użytkownika i zapewnia spójność pomiędzy różnymi sekcjami aplikacji.

II.3.2. Architektura kodu warstwy logiki biznesowej

Django został zaprojektowany w oparciu o wzorzec MVT (Model-View-Template) [6], który zakłada wyraźny podział na warstwę danych, logikę biznesową oraz prezentację informacji użytkownikowi. W przypadku tej aplikacji Django pełni funkcję wyłącznie backendu. Warstwa logiki biznesowej w tej architekturze odpowiada za wszystkie reguły, procesy i przepływ danych pomiędzy warstwą danych a interfejsem użytkownika. Jej zadaniem jest zarządzanie działaniem systemu niezależnie od tego, w jaki sposób dane są prezentowane na stronie. Warstwa biznesowa jest rozdzielona na kilka elementów:

- Widoki (views.py) – Pełnią kluczową rolę w realizacji logiki biznesowej, odpowiadając za obsługę operacji przychodzących od użytkownika. W widokach następuje interpretacja żądań pochodzących z interfejsu użytkownika (np. dodanie transakcji, edycja celu, spłata zobowiązania), walidacja danych wejściowych oraz podjęcie decyzji, które operacje powinny być wykonane na modelach. Widoki odpowiadają również za sprawdzenie uprawnień oraz obsługę wyjątków, co zwiększa bezpieczeństwo i stabilność systemu.
- Serializery (serializers.py) – Umożliwiają konwertowanie danych pomiędzy wewnętrznymi strukturami aplikacji a formatem (JSON) wykorzystywanym podczas komunikacji z interfejsem użytkownika. Odpowiadają także za walidację i weryfikację poprawności przesyłanych informacji, co zapewnia bezpieczeństwo i spójność danych.
- Modele (models.py) – Głównym zadaniem modeli jest opis struktury danych oraz reguł obowiązujących w systemie. Definiują one jakie informacje są gromadzone oraz w jaki sposób poszczególne obiekty pozostają ze sobą powiązane. Zawierają one również metody, które umożliwiają bezpośrednie wykonywanie operacji na danych.
- Ścieżki (urls.py) – Plik ścieżek określa, pod jakimi adresami (endpointami) dostępne są poszczególne funkcjonalności systemu. Każda ścieżka jest powiązana z określonym widokiem, który odpowiada za realizację właściwych operacji biznesowych.

Oprócz opisanych wyżej podstawowych komponentów, w aplikacji wykorzystano również dodatkowe rozwiązania wspierające logikę biznesową, które nie należą do klasycznego trzonu Django:

- Moduł prognozowania finansowego – aplikacja zawiera osobny plik odpowiedzialny za operacje związane z przewidywaniem przyszłych

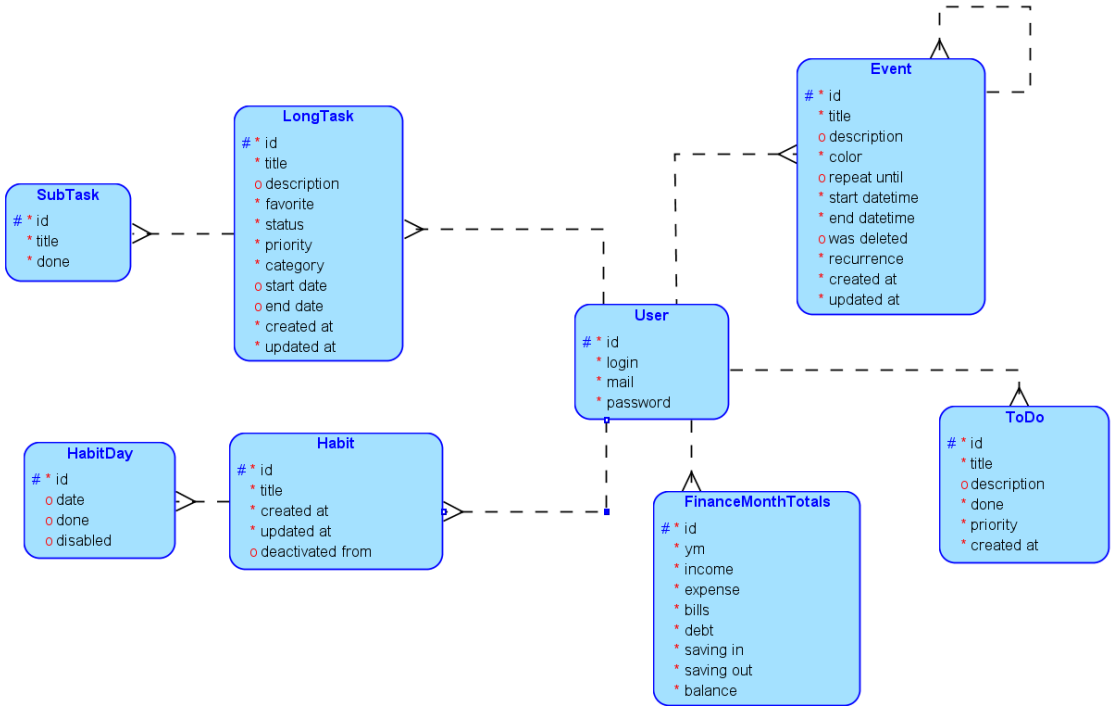
przychodów i wydatków użytkownika. Moduł ten analizuje zgromadzone dane historyczne i na tej podstawie przygotowuje prognozę na kolejne miesiące, które wyświetlają się w interfejsie aplikacji.

II.3.3. Architektura kodu warstwy danych

Architektura kodu warstwy danych jest zrealizowana zgodnie z podejściem stosowanym w Django, gdzie główną rolę pełnią modele. Dzięki systemowi migracji wszelkie zmiany w modelach są automatycznie odwzorowane na strukturze danych. Dodatkowo, modele są podstawą dla działania pozostałych warstw aplikacji. To na nich opiera się zarówno logika biznesowa, jak i warstwa prezentacji, która używa danych zgromadzonych i przetwarzanych przez te struktury.

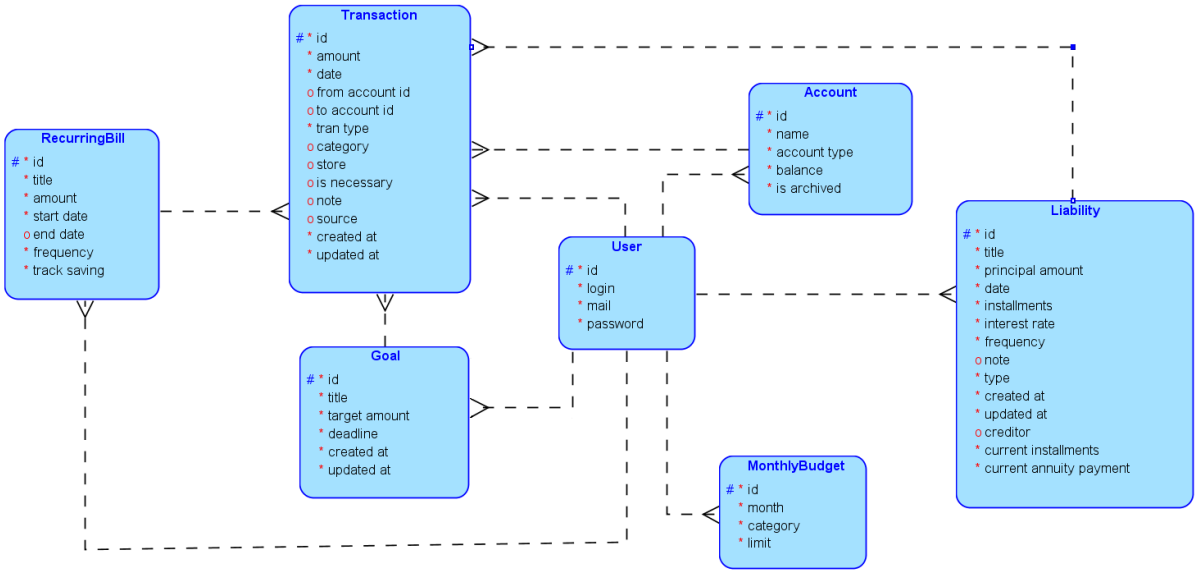
II.4. Model logiczny danych

W tym podrozdziale przedstawiono logiczną reprezentację danych opracowaną w środowisku „Oracle SQL Developer Data Modeler". Diagram odwzorowuje strukturę bazy danych w postaci encji z ich atrybutami, wskazując jednoznacznie klucze główne oraz powiązania referencyjne między tabelami (Rys.5, Rys.6). Atrybuty zostały oznaczone pod kątem wymagalności danych, co umożliwia identyfikację pól obowiązkowych i opcjonalnych. Diagram systemu został podzielony na dwie główne części, dla zachowania porządku w strukturze danych. Centralnym elementem całego modelu jest encja użytkownika (User), która stanowi punkt odniesienia dla wszystkich pozostałych obiektów w systemie. Wszystkie kluczowe encje korzystają z relacji wiele-do-jednego względem użytkownika, dzięki czemu każdy użytkownik zarządza wyłącznie swoimi danymi.



Rys. 5. Model danych część 1

Źródło: Opracowanie własne.



Rys. 6. Model danych część 2

Źródło: Opracowanie własne.

- Model użytkownika (User) - to punkt odniesienia dla wszystkich pozostałych danych w aplikacji. Przechowuje podstawowe dane logowania, takie jak login, adres e-mail oraz zahaszowane hasło. Każdy użytkownik posiada własny zestaw kont, transakcji, zadań, nawyków, celów, zobowiązań, wydarzeń kalendarza i miesięcznych podsumowań. Dzięki temu system umożliwia pełną separację i bezpieczeństwo danych indywidualnych użytkowników.
- Model zadań krótkich (ToDo) - służy do zarządzania prostymi zadaniami codziennymi użytkownika. Każde zadanie przechowuje tytuł, opcjonalny opis, status wykonania (czy zostało ukończone), priorytet oraz datę utworzenia. Zadania są przypisane do konkretnego użytkownika i służą do śledzenia bieżących spraw, które użytkownik chce zrealizować w krótkim czasie.
- Model zadań długich (LongTask) - opisuje rozbudowane, długoterminowe zadania lub projekty, które mogą być rozbite na mniejsze podzadania (SubTask). Przechowuje szczegółowe informacje, takie jak tytuł, opis, ulubione, status, priorytet, kategoria oraz daty rozpoczęcia i zakończenia. Podzadania to zadania cząstkowe należące do zadań długich, z własnym tytułem i flagą wykonania. Dzięki temu użytkownik może planować większe projekty i śledzić postęp ich realizacji.
- Model nawyków (Habit) - umożliwia użytkownikowi śledzenie nawyków (np. regularnych ćwiczeń, nauki). Model ten przechowuje nazwę nawyku, datę utworzenia, modyfikacji oraz ewentualnej dezaktywacji. Z kolei dzień nawyku (HabitDay) przechowuje informacje o realizacji danego nawyku w konkretnym dniu i czy nie był wyłączony. Pozwala to na łatwą analizę regularności oraz postępów w budowaniu nawyków.

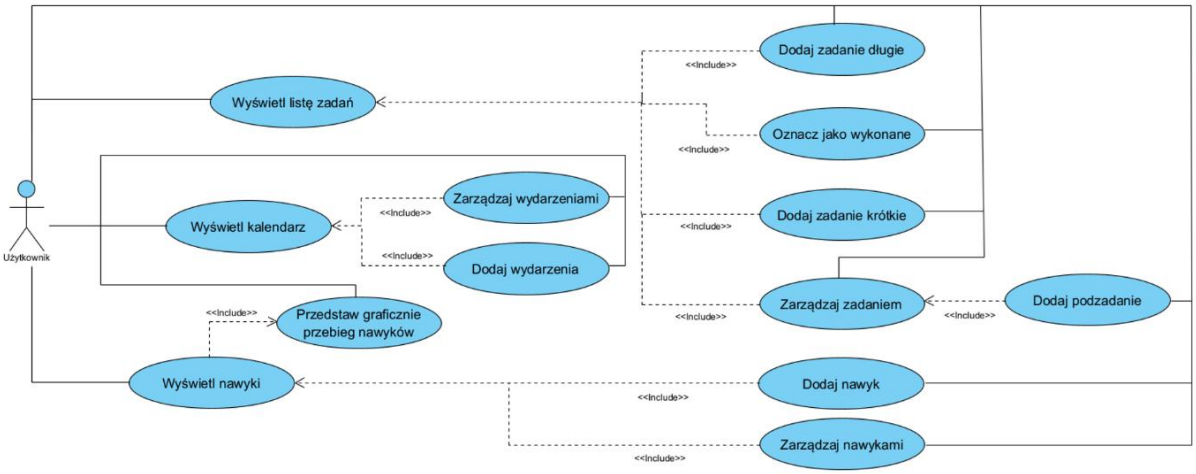
- Model wydarzenia (Event) - opisuje wydarzenia w kalendarzu użytkownika, od jednorazowych wydarzeń, przez cykliczne zadania, po różnego typu przypomnienia czy terminy. Model przechowuje tytuł, opis, kolor oznaczenia, zakres czasowy (data i godzina rozpoczęcia oraz zakończenia), informacje o powtarzalności oraz status (np. czy wydarzenie zostało usunięte).
- Model podsumowania miesięcznego finansów (FinanceMonthTotals) - opisuje podsumowania finansowe użytkownika w skali miesiąca. Przechowuje informacje takie jak łączna suma przychodów, wydatków, kwoty przeznaczone na rachunki, oszczędności, zadłużenie oraz saldo na koniec miesiąca. Pozwala to na szybkie zestawienia, analizę historii finansowej i kontrolowanie budżetu na przestrzeni czasu.
- Model kont finansowych (Account) - opisuje konta finansowe użytkownika, takie jak konto główne, oszczędnościowe czy konto techniczne. Przechowuje nazwę konta, typ (np. gotówkowe, oszczędnościowe), bieżące saldo oraz informację, czy konto jest aktywne czy zarchiwizowane. Umożliwia to rozdzielenie środków według potrzeb i większą kontrolę nad własnymi finansami.
- Model transakcji finansowych (Transaction) - to kluczowy model dla rejestracji wszystkich operacji finansowych użytkownika. Przechowuje kwotę, datę, powiązania z kontem źródłowym i docelowym (np. przy przelewach), kategorię (np. jedzenie, transport), typ transakcji (przychód, wydatek, transfer), a także szczegóły takie jak sklep, notatka czy informacja o konieczności wydatku. Każda transakcja przypisana jest do użytkownika i w razie potrzeby powiązana z celem oszczędnościowym, zobowiązaniem lub cyklicznym rachunkiem.
- Model celów finansowych (Goal) - służy do tworzenia celów oszczędnościowych użytkownika (np. odkładania środków na wakacje czy większy zakup). Model zawiera tytuł, docelową kwotę, termin realizacji oraz daty utworzenia i modyfikacji. Powiązanie z transakcjami pozwala rejestrować wpłaty i wypłaty związane z danym celem.
- Model zobowiązań finansowych (Liability) - to model odpowiedzialny za zobowiązania finansowe użytkownika, takie jak kredyty, pożyczki czy inne długi. Przechowuje informacje o kwocie głównej do spłaty, dacie rozpoczęcia, liczbie rat, oprocentowaniu, częstotliwości spłat, notatkach, typie zobowiązania, wierzycielu oraz obecnym stanie rozliczenia. Kluczowym aspektem jest powiązanie każdego zobowiązania z odpowiednimi transakcjami finansowymi, które rejestrują konkretne spłaty rat. Pozwala to użytkownikowi na pełną kontrolę nad procesem spłaty zobowiązań i monitorowanie terminów rat.
- Model rachunków (RecurringBill) - służy do zarządzania cyklicznymi opłatami i rachunkami (np. abonamenty, rachunki za prąd czy OC). Model przechowuje tytuł rachunku, kwotę, terminy początkowy i końcowy, częstotliwość oraz

informację, czy należy odkładać środki z wyprzedzeniem na daną opłatę. Powiązanie rachunków z odpowiednimi transakcjami jest kluczowe, ponieważ każda opłata lub odłożenie pieniędzy na przyszły rachunek odnotowywana jest jako osobna transakcja finansowa. Dzięki temu użytkownik nie przeoczy żadnej ważnej płatności.

- Model budżetu miesięcznego (MonthlyBudget) pozwala użytkownikowi ustalać limity wydatków w wybranych kategoriach na dany miesiąc. Przechowuje informację o miesiącu, kategorii oraz ustalonym limicie. Funkcjonalność ta wspiera świadome zarządzanie budżetem i pomaga trzymać finanse pod kontrolą.

II.5. Diagram przypadków użycia

Diagram przypadków użycia to sposób graficznego przedstawienia interakcji między aktorami a funkcjami dostępnymi w systemie. Pokazuje on strukturę systemu z punktu widzenia użytkownika, koncentrując się na tym, jakie działania można wykonać oraz kto ma do nich dostęp. Tego typu diagram jest przydatny na etapie analizy wymagań, ponieważ pomaga w identyfikacji kluczowych funkcjonalności, planowaniu implementacji i weryfikacji kompletności systemu. Ze względu na szeroki zakres funkcjonalności projektowanej aplikacji, przypadki użycia zostały rozdzielone na dwa osobne diagramy. Pierwszy z nich obejmuje wszystkie operacje związane z zarządzaniem zadaniami, nawykami oraz kalendarzem, czyli te funkcje, które wspierają użytkownika w organizacji czasu, planowaniu oraz budowaniu nawyków. Drugi diagram koncentruje się wyłącznie na funkcjach finansowych.

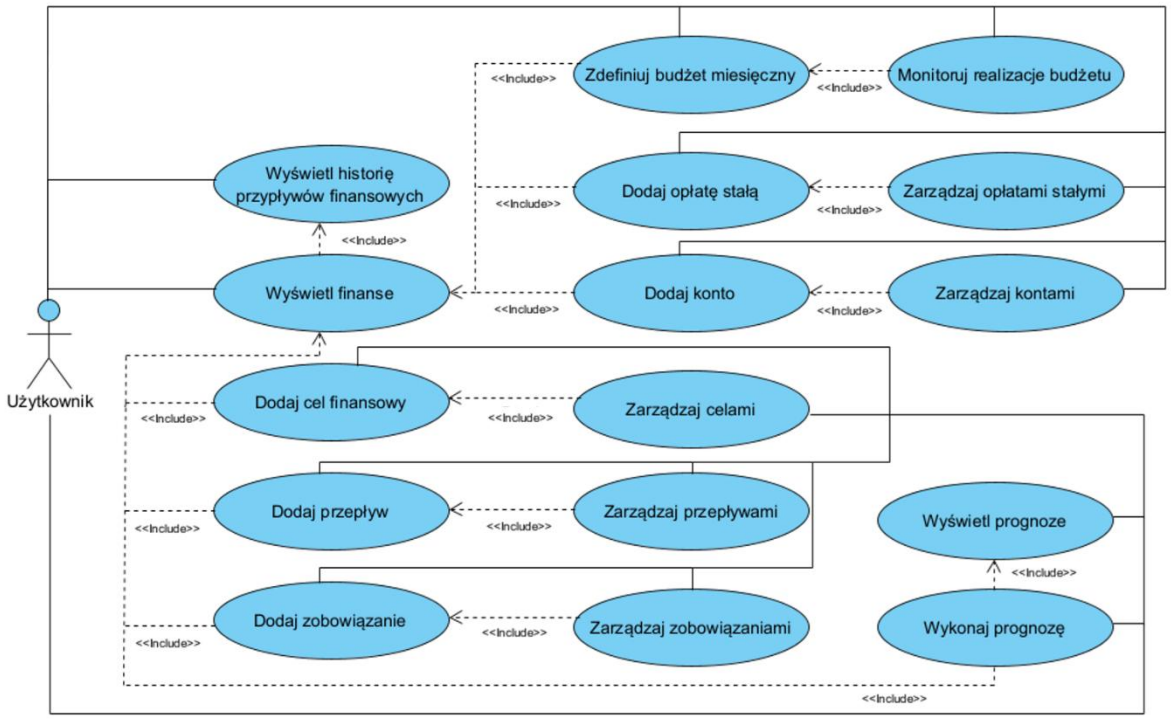


Rys. 7. Diagram przypadków użycia dla zadań, wydarzeń i nawyków

Źródło: Opracowanie własne.

Pierwszy diagram przypadków użycia przedstawia obsługę modułów związanych z zadaniami, kalendarzem oraz nawykami (Rys.7). Najważniejszą cechą tego diagramu są relacje typu <<include>>, które wskazują, że podstawowe przypadki użycia zawierają dodatkowe, bardziej szczegółowe operacje, niezbędne do ich pełnej realizacji.

- W przypadku „Wyświetl listę zadań” przypadek użycia zawsze zawiera operacje takie jak dodanie zadania krótkiego, zadania długiego, oznaczenie zadania jako wykonane oraz zarządzanie zadaniami długimi. Ten ostatni przypadek użycia obejmuje również możliwość dodawania podzadań. Relacje <<include>> wskazują, że każda z tych operacji stanowi integralną część funkcjonalności dostępnych w ramach przeglądania listy zadań,
- Podobnie w przypadku kalendarza, przypadek użycia „Wyświetl kalendarz” zawiera operacje takie jak dodawanie wydarzeń oraz zarządzanie nimi. Relacje <<include>> wskazują, że czynności te stanowią naturalne rozszerzenie podstawowego widoku kalendarza i są dostępne bezpośrednio po jego otwarciu.
- W module nawyków przypadek użycia „Wyświetl nawyki” obejmuje możliwość dodawania i zarządzania nawykami. Dodatkowo, dzięki relacji <<include>>, użytkownik może od razu zobaczyć graficzne przedstawienie postępów, co jest integralną częścią tego widoku.



Rys. 8. Diagram przypadków użycia dla finansów

Źródło: Opracowanie własne.

Diagram drugi przedstawia wszystkie kluczowe przypadki użycia związane z zarządzaniem finansami osobistymi użytkownika (Rys. 8). Centralny przypadek „Wyświetl finanse” stanowi punkt wyjścia dla większości operacji dostępnych w systemie. Pozostałe przypadki użycia są z nim powiązane poprzez relacje <<include>>, co oznacza, że są one integralną częścią modułu finansowego i mogą być wywoływane w ramach korzystania z głównej funkcji przeglądania finansów.

- „Dodaj konto” to operacja pozwalająca użytkownikowi utworzyć nowe konto finansowe w systemie. Dopiero po jego utworzeniu dostępny staje się przypadek „Zarządzaj kontem” (<<include>>), umożliwiający edycję, archiwizację, transfer środków lub usunięcie wybranego konta finansowego. Bez stworzonego konta finansowego nie ma możliwości zarządzania nim.
- „Dodaj opłatę stałą” pozwala użytkownikowi na zdefiniowanie nowego rachunku cyklicznego (np. abonament, rachunek za prąd), natomiast przypadek „Zarządzaj opłatą stałą” (<<include>>) umożliwia pełną obsługę tej opłaty (np. edycja, historia płatności) po jego dodaniu.
- Funkcja „Zdefiniuj budżet miesięczny” pozwala użytkownikowi określić limity wydatków w wybranych kategoriach na dany miesiąc. Po ustawieniu budżetu aktywuje się przypadek „Monitoruj realizację budżetu” (<<include>>), umożliwiający bieżącą kontrolę przestrzegania założonych limitów oraz analizę, czy użytkownik mieści się w założonym planie.
- Funkcja „Dodaj cel finansowy” pozwala na stworzenie nowego celu do oszczędzania, natomiast przypadek „Zarządzaj celem finansowym” (<<include>>) jest dostępny wyłącznie dla już zdefiniowanych celów. Umożliwia on monitorowanie postępów, edycję, wpłaty, wypłaty czy usunięcie celu, dzięki czemu użytkownik ma pełną kontrolę nad realizacją swoich zamierzeń.
- "Przepływy finansowe" - użytkownik może rejestrować wszystkie swoje transakcje, zarówno wydatki, jak i przychody, korzystając z funkcji „Dodaj przepływ”. Szczegółowe zarządzanie każdą z tych operacji, takie jak analiza czy usunięcie, jest możliwe po ich dodaniu i realizowane jest poprzez przypadek „Zarządzaj przepływem” (<<include>>).
- Przypadek „Dodaj zobowiązanie” umożliwia rejestrację nowego zadłużenia, a „Zarządzaj zobowiązaniem” (<<include>>) otwiera możliwość m.in. wprowadzania spłat, edycji czy podglądu szczegółów zadłużenia, ale tylko w odniesieniu do wcześniej wprowadzonych zobowiązań.
- "Wyświetl historię przepływów finansowych" – użytkownik może przeglądać wszystkie transakcje i zawężać wyniki według wybranych kryteriów.
- Przypadek „Wyświetl prognozę” pozwala użytkownikowi przejść do sekcji analizy przewidywanych przychodów i wydatków. Wywołuje on automatycznie przypadek „Wykonaj prognozę” (<<include>>), ponieważ każdorazowe wyświetlenie prognozy wymaga wygenerowania aktualnych danych analitycznych. Taka konstrukcja podkreśla, że prezentacja prognozy jest nierozerwalnie związana z jej wcześniejszym obliczeniem.

Rozdział III. Implementacja funkcjonalności

III.1. Narzędzia programistyczne i środowisko

Do implementacji systemu wykorzystano środowisko PyCharm, [7] narzędzie od firmy JetBrains zbudowane na platformie IntelliJ IDEA, które zostało stworzone z myślą o pracy z Pythonem. Choć PyCharm jest ukierunkowany na rozwój aplikacji w Pythonie, dzięki dużej ilości dostępnych wtyczek obsługuje również języki i technologie pokrewne, m.in. JavaScript czy SQL.

Wbudowane wsparcie dla Django bardzo ułatwia pracę, można korzystać z kreatorów widoków czy przeglądać bazę danych bezpośrednio w IDE. Mechanizmy inteligentnego autouzupełniania oraz statycznej analizy znacząco przyspieszają i ułatwiają pracę, pomagając utrzymać wysoką jakość kodu. PyCharm oferuje ponadto narzędzia przydatne w projektach webowych i danych takie jak: kreatory szablonów Django i Flask.

Przyjazny interfejs w połączeniu z bogatym zestawem funkcji sprawiają, że PyCharm znacząco usprawnia proces tworzenia aplikacji w Pythonie, oferując jednocześnie elastyczność wymaganą przy pracy z nowoczesnymi frameworkami i narzędziami.

III.2. Wykorzystane technologie

W niniejszym podrozdziale przedstawiono zestaw technologii programistycznych, które zostały wykorzystane podczas realizacji aplikacji. Wybór narzędzi został podyktowany ich popularnością, elastycznością oraz dostępnością szerokiego ekosystemu bibliotek i wsparcia społeczności. Aplikacja została zbudowana z użyciem języków i technologii typowych dla aplikacji webowych: Python i JavaScript odpowiadają za logikę aplikacji, natomiast HTML i CSS służą do budowy i stylizacji interfejsu użytkownika. Do przechowywania danych wykorzystano relacyjny system bazodanowy PostgreSQL.

III.2.1. Python

Python [8] to język programowania zaprojektowany z myślą o prostocie, przejrzystości i czytelności kodu. Został opracowany przez Guido van Rossuma na przełomie lat 80. i 90., a jego pierwsza oficjalna wersja pojawiła się w 1991 roku.

Jedną z największych zalet Pythona jest jego składnia, zbliżona do języka naturalnego, co czyni go łatwym do nauki i przyjaznym dla początkujących programistów. Język ten należy do języków wysokopoziomowych, a jego możliwości są wspierane przez rozbudowaną bibliotekę standardową oraz ekosystem zewnętrznych pakietów.

Python znajduje praktyczne zastosowanie w tworzeniu aplikacji webowych, w analizie danych, statystyce i uczeniu maszynowym korzystając z bibliotek takich jak:

NumPy, pandas, scikit-learn, co było przydatne podczas tworzenia predykcji finansowych. W aplikacji Python został użyty jako główny język programistyczny po stronie serwera.

III.2.2. Django

Django [9] to wysokopoziomowy framework webowy napisany w języku Python, zaprojektowany z myślą o szybkim tworzeniu aplikacji internetowych oraz zachowaniu przejrzystej, czytelnej struktury kodu. Rozwijany jest jako projekt otwartoźródłowy pod opieką Django Software Foundation.

Najważniejsze atuty frameworka to: wbudowany, intuicyjny interfejs administracyjny generowany automatycznie z definicji modeli, czytelne operacje na bazach relacyjnych, wbudowane mechanizmy uwierzytelniania, obsługa migracji schematu bazy, cache i sesje.

W projekcie Django został wykorzystany wyłącznie do obsługi logiki biznesowej, autoryzacji użytkowników i komunikacji z bazą danych PostgreSQL. Dzięki posiadaniu ORM można w szybki sposób sprawnie modelować danymi, zamiast pisania zapytań SQL.

Rozszerzenie Django REST Framework pozwoliło w prosty sposób stworzyć API, przez które cała komunikacja z frontendem odbywa się wyłącznie poprzez zapytania HTTP. Dzięki temu możliwe jest sprawne zarządzanie danymi, kontrolowanie dostępu oraz obsługa różnych operacji po stronie klienta.

III.2.3. JavaScript

JavaScript to język programowania wykorzystywany do tworzenia dynamicznych i interaktywnych elementów w aplikacjach internetowych. W projekcie odpowiada za obsługę interfejsu użytkownika, umożliwia np. aktualizację listy zadań, kont, czy transakcji bez przeładowywania strony. Wykorzystano go również do akceptacji formularzy, obsługi kliknięć, otwierania i zamykania modali oraz do komunikacji z backendem przy użyciu Fetch API. Dzięki temu aplikacja reaguje na działania użytkownika natychmiastowo i zapewnia płynne, nowoczesne doświadczenie korzystania z systemu. JavaScript umożliwił też implementację modali, przełączników, obsługi kliknięć oraz wielu elementów interfejsu, które reagują w czasie rzeczywistym, co sprawiło, że aplikacja działa płynnie i jest bardziej przyjazna dla użytkownika. Dodatkowo za pomocą JavaScript zostały dodane interaktywne wykresy pokazujące dane oraz paski postępu.

III.2.4. HTML

HTML to język znaczników, który odpowiada za tworzenie struktur stron internetowych i prezentacji dokumentów w sieci WWW. Struktura dokumentu HTML jest hierarchiczna i oparta na zagnieżdżeniu elementów, co pozwala na logiczne

uporządkowanie treści i ułatwia ich interpretację zarówno przez przeglądarki, jak i inne narzędzia przetwarzające dane.

W opracowywanej aplikacji HTML pełni rolę szkieletu interfejsu użytkownika, definiuje strukturę szablonów ładowanych przez przeglądarkę oraz komponentów dynamicznie tworzonych przez JavaScript. To właśnie on określa, co gdzie się znajduje i w jakiej kolejności użytkownik widzi poszczególne elementy.

III.2.5. CSS

CSS to technologia pozwalająca na rozdzielenie warstwy prezentacji graficznej od struktury logicznej dokumentu w językach znaczników, takich jak HTML. W kontekście tworzenia stron internetowych CSS pełni kluczową rolę w definiowaniu estetyki oraz układu treści. Technologia opiera się na mechanizmie kaskadowości, co pozwala na hierarchiczne nakładanie się stylów, gdzie reguły bardziej szczegółowe mają pierwszeństwo nad ogólnymi. Style mogą być osadzone bezpośrednio w kodzie HTML lub umieszczone w zewnętrznych plikach.

CSS odpowiada w aplikacji za wygląd interfejsu czyli kolory, rozmieszczenie elementów, odstępy, czcionki i ogólny styl strony. Dzięki zewnętrznym arkuszom stylów udało się utrzymać spójny wygląd całego systemu. CSS pozwolił na stworzenie układu oraz dopracowanie detali graficznych i estetycznych. Styl został oddzielony od struktury HTML co ułatwiło dalsze modyfikacje i utrzymanie przejrzystości kodu frontendowego.

III.2.6. PostgreSQL

PostgreSQL [10] to obiektowo-relacyjny system zarządzania bazą danych, który powstał na Uniwersytecie Kalifornijskim w Berkeley i od lat jest rozwijany przez globalną społeczność. Charakteryzuje się wysoką skalowalnością, niezawodnością oraz zgodnością ze standardem SQL. Znajduje zastosowanie w systemach przetwarzania dużych zbiorów danych, aplikacjach webowych czy systemach analitycznych.

W naszej aplikacji PostgreSQL pełni rolę głównej bazy danych. Przechowuje wszystkie informacje, w tym dane użytkowników.

III.3. Wykorzystane biblioteki

III.3.1. Statsmodels

W projekcie do realizacji części odpowiedzialnej za prognozowanie finansów użytkownika wykorzystano bibliotekę statsmodels [11] w wersji 0.14.4. Statsmodels jest jedną z kluczowych bibliotek języka Python, przeznaczoną specjalnie do modelowania statystycznego i analizy danych. Statsmodels udostępnia gotowe narzędzia do budowy regresji liniowych, pracy z szeregami czasowymi, przeprowadzania testów statystycznych i estymacji modeli ekonometrycznych. W aplikacji biblioteka ta została wykorzystana do budowania prognoz przychodów

i wydatków użytkownika na kolejne miesiące na podstawie danych zebranych z historii transakcji.

Dzięki wygodnemu podejściu opartemu na metodach „fit()”, „predict()” i „summary()”, tworzenie modeli i odczytywanie wyników było szybkie i przejrzyste. Dodatkowo, dobra integracja statsmodels z bibliotekami NumPy i pandas znacznie ułatwiła przetwarzanie danych i pracę nad kodem.

III.3.2. Psycopg2-binary

Do zarządzania połączeniem z bazą danych PostgreSQL oraz wykonywania zapytań SQL w projekcie wykorzystano bibliotekę psycopg2-binary [12] w wersji 2.9.10. Jest to najpopularniejszy adapter bazy danych PostgreSQL dla języka Python, przeznaczony do pracy z systemem PostgreSQL. Jej głównym zadaniem jest realizacja interfejsu umożliwiającego wykonywanie zapytań SQL, zarządzanie połączeniami z bazą oraz wydajne operowanie na danych. Zaprojektowano go z myślą o aplikacjach, które intensywnie korzystają z wielu wątków, czy wykonują dużą liczbę jednoczesnych operacji, takich jak INSERT lub UPDATE na bazie danych.

Biblioteka psycopg2-binary zapewnia wysoką wydajność oraz niezawodność, a także zgodność z oficjalnym standardem Python DB-API 2.0. Dzięki temu możliwa jest efektywna wymiana danych pomiędzy aplikacją a bazą danych PostgreSQL. Wykorzystanie tej biblioteki usprawnia realizację części serwerowej aplikacji oraz zapewnia bezpieczną i stabilną komunikację między warstwami.

III.3.3. NumPy

W projekcie wykorzystano bibliotekę NumPy [13] w wersji 2.2.5, która jest jednym z podstawowych narzędzi języka Python do pracy z danymi numerycznymi i wykonywania obliczeń matematycznych. NumPy zapewnia efektywne struktury danych, w szczególności tablice wielowymiarowe, które umożliwiają szybkie przeprowadzanie operacji arytmetycznych i matematycznych na dużych zbiorach danych. Dzięki swoim zoptymalizowanym implementacjom, biblioteka ta znacząco przyspiesza obliczenia numeryczne, które są kluczowe w realizacji części analityczno-predykcyjnej niniejszej pracy.

NumPy odgrywa istotną rolę w integracji z innymi bibliotekami analitycznymi, takimi jak statsmodels czy pandas, stanowiąc ich zaplecze obliczeniowe. Użycie NumPy pozwala nie tylko na sprawne manipulowanie danymi historycznymi, ale również na efektywne przygotowanie ich do modelowania statystycznego i prognozowania. W projekcie okazał się niezbędny do sprawnego przetwarzania liczb i przygotowywania danych do dalszej analizy.

III.3.4. Chart.js

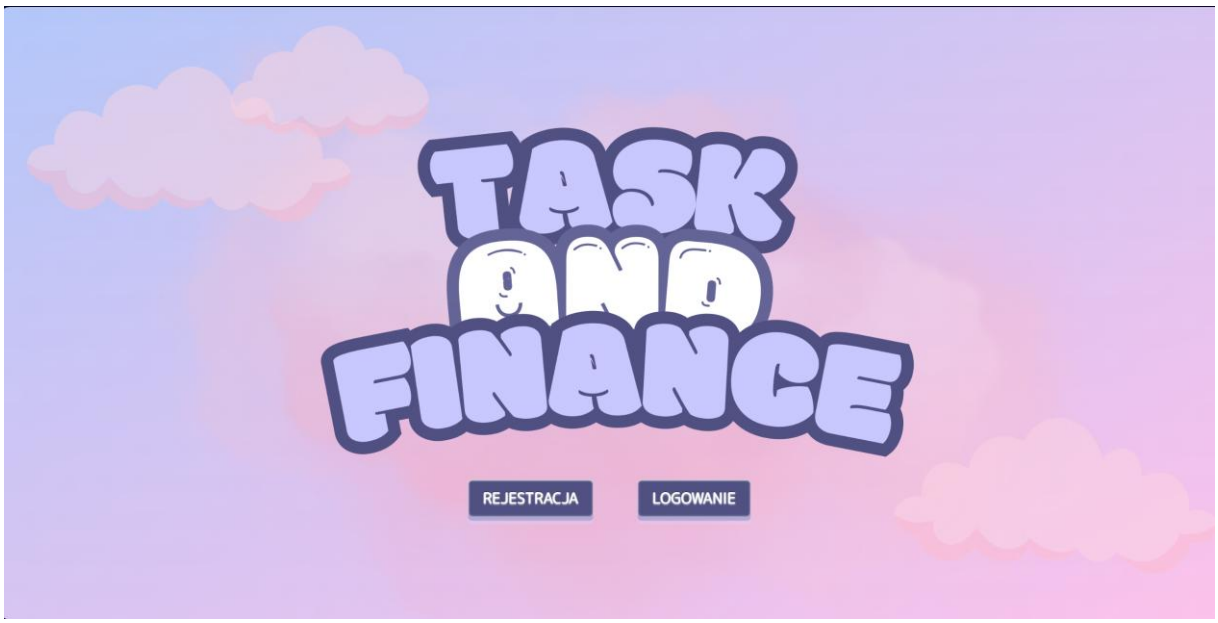
Do wizualizacji danych w warstwie interfejsu użytkownika zastosowano bibliotekę Chart.js [14] w wersji 4.4.9. Jest to elastyczna biblioteka JavaScript, która pozwala na efektywne generowanie interaktywnych i estetycznych wykresów oraz diagramów bezpośrednio w przeglądarce użytkownika. Chart.js posiada zestaw gotowych typów wykresów, takich jak: wykresy liniowe, słupkowe, kołowe oraz radarowe, które umożliwiają przejrzyste i intuicyjne przedstawianie wyników analizy danych.

Ponadto w celu wzbogacenia wizualizacji oraz wskazania istotnych punktów odniesienia na wykresach, zastosowano dodatek chartjs-plugin-annotation w wersji 3.1.0. Plugin, który umożliwia tworzenie adnotacji oraz oznaczeń bezpośrednio na wykresach, takich jak linie odniesienia czy dodatkowe etykiety opisowe. Dzięki integracji Chart.js z tym rozszerzeniem możliwe było uzyskanie bardziej czytelnych wykresów dzięki dodatkowym szczegółom naniesionym na wykres.

III.4. Prezentacja aplikacji

W tym podrozdziale zaprezentowane zostaną kluczowe elementy interfejsu użytkownika aplikacji wraz z ich funkcjonalnością. Dla każdego widoku przedstawiono wygląd graficzny oraz opisano jego sposób działania w kontekście całej aplikacji. Dla wybranych funkcji przedstawiono opis implementacji.

III.4.1. Ekran startowy



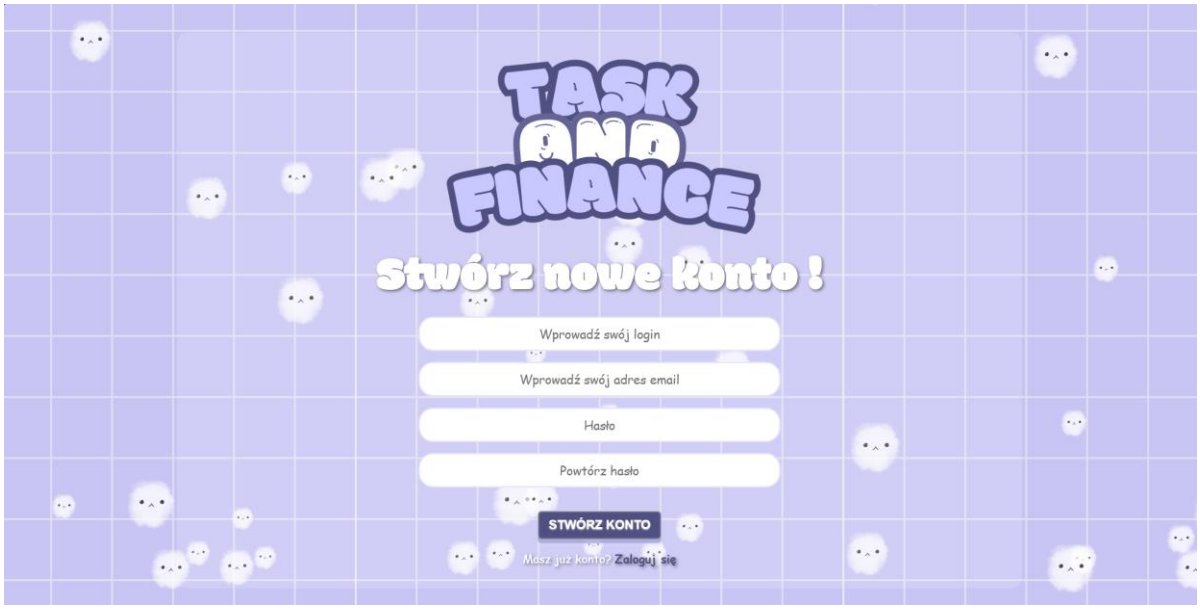
Rys. 9. Strona startowa aplikacji

Źródło: Opracowanie własne.

Po uruchomieniu aplikacji użytkownik zostaje przeniesiony do strony startowej, która pełni funkcje ekranu powitalnego oraz punktu wyjścia do dalszego korzystania

z systemu (Rys. 9). Widok ten przedstawia nazwę aplikacji oraz oferuje możliwość przejścia na stronę rejestracji lub logowania.

III.4.2. Ekran rejestracji

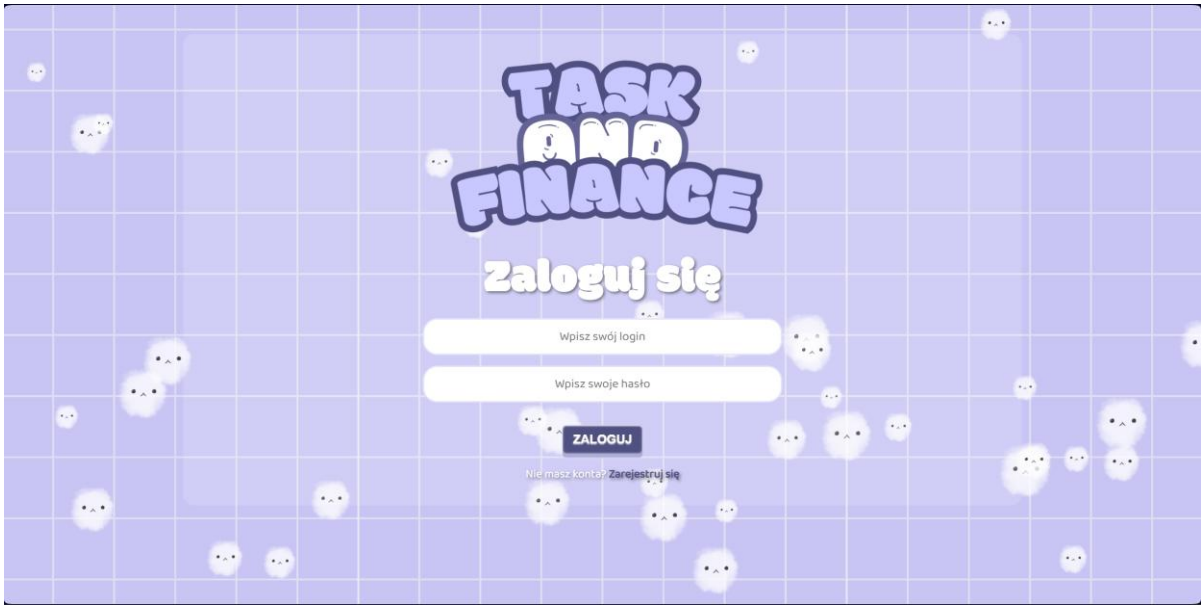


Rys. 10. Strona rejestracji aplikacji

Źródło: Opracowanie własne.

Kolejnym widokiem jest strona umożliwiająca użytkownikowi rejestrację nowego użytkownika (Rys. 10). Głównym elementem interfejsu jest formularz umożliwiający podanie unikalnej nazwy użytkownika. Aplikacja wykorzystuje standardowy system użytkowników Django. Podczas procesu rejestracji użytkownika są wykorzystywane wbudowane mechanizmy frameworka Django do bezpiecznego przechowywania haseł. Hasła użytkowników nigdy nie są przechowywane w jawnej formie, są one przetwarzane przy pomocy silnych algorytmów kryptograficznych PBKDF2 z SHA256 generując unikalne dla każdego użytkownika skróty hashowe. Dzięki temu nawet w przypadku uzyskania nieautoryzowanego dostępu do bazy danych, odtworzenie hasła użytkownika jest praktycznie niemożliwe.

III.4.3. Ekran logowania



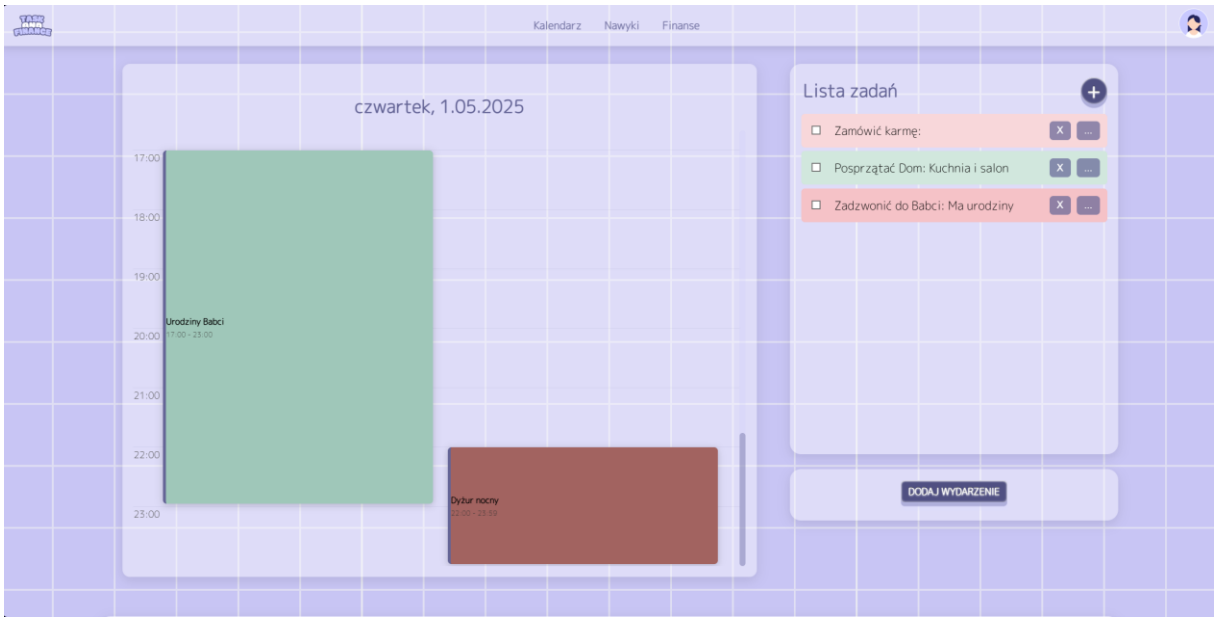
Rys. 11. Strona logowania aplikacji

Źródło: Opracowanie własne.

Po przejściu na ekran logowania użytkownik poproszony jest o podanie nazwy użytkownika oraz hasła (Rys. 11). Po naciśnięciu przycisku „zaloguj” dane są przekazywane do backendu, następnie na serwerze funkcja uwierzytelniania sprawdza, czy dane są poprawne, poprzez porównanie podanego hasła z jego skrótem (hashem) zapisanym w bazie danych. Po pozytywnej weryfikacji, zostaje zapisany identyfikator użytkownika sesji i dzięki temu uzyskuje on dostęp do zabezpieczonych zasobów aplikacji, chronionych dekoderami.

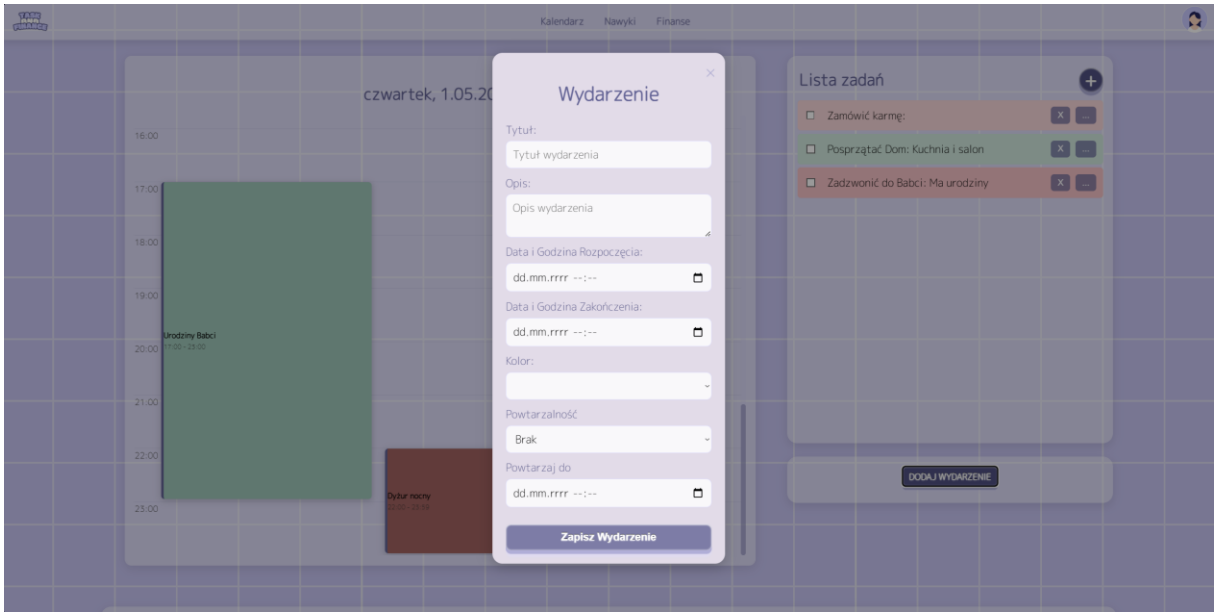
Dekorator permission_classes([IsAuthenticated]) został wykorzystany we wszystkich widokach API, które wymagają uwierzytelnienia. Dzięki temu dostęp do danych jest możliwy wyłącznie dla zalogowanych użytkowników, posiadających aktywną sesje. Próba uzyskania dostępu bez autoryzacji skutkuje natychmiastową odmową realizacji zapytania przez serwer. Wylogowanie natychmiast unieważnia token sesji, minimalizując ryzyko nieautoryzowanego dostępu przy opuszczeniu aplikacji przez użytkownika.

III.4.4. Ekran kalendarza



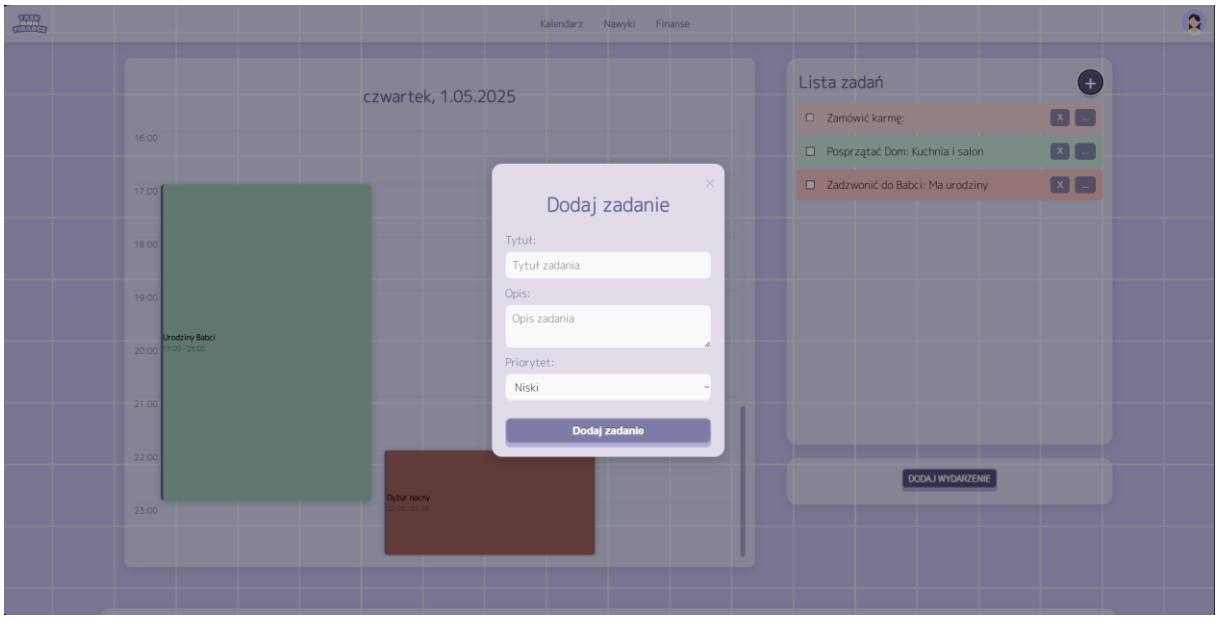
Rys. 12. Kalendarz i listą zadań krótkich

Źródło: Opracowanie własne.



Rys. 13. Formularz wydarzenia

Źródło: Opracowanie własne.



Rys. 14. Formularz zadania krótkiego

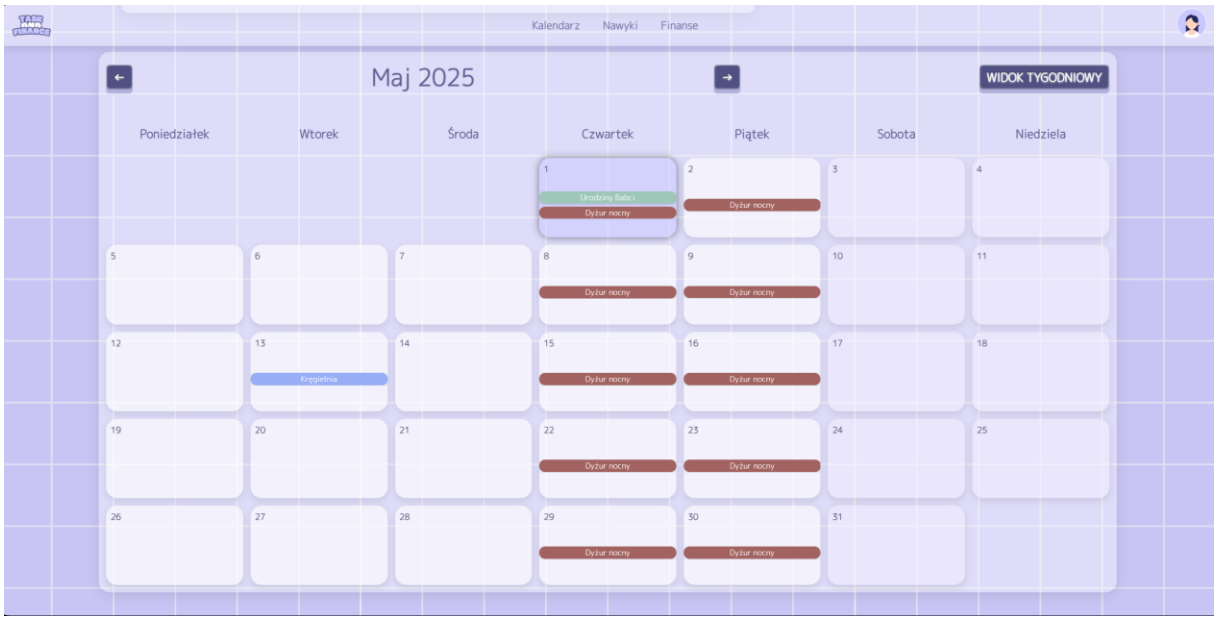
Źródło: Opracowanie własne.

Po zalogowaniu użytkownik otrzymuje widok kalendarza, który umożliwia przeglądanie i zarządzanie wydarzeniami oraz zadaniami (Rys. 12). Po wybraniu konkretnego dnia z kalendarza miesięcznego, użytkownik przechodzi do szczegółowego widoku dnia, który prezentuje wszystkie zaplanowane wydarzenia w formie bloków rozmieszczonych na osi godzinowej. Użytkownik może w łatwy sposób podejrzeć szczegóły wydarzenia, poprzez kliknięcie na nie. Otrzymuje wtedy też możliwość edycji wydarzeniu lub jego usunięcia.

Po prawej stronie widoczna jest lista zadań krótkich, dzięki niej użytkownik może w prosty sposób codziennie definiować zadania, a także, odznaczyć lub usunąć. Pomaga to w szybkim zdefiniowaniu rzeczy do zrobienia na dany dzień.

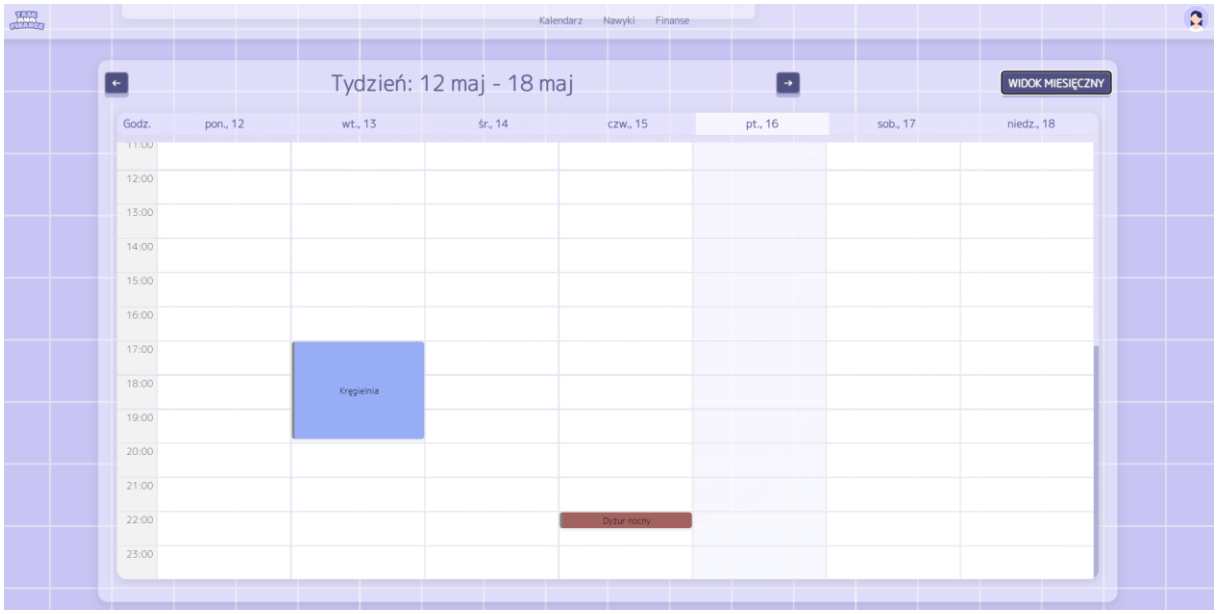
Formularz wydarzenia umożliwia podanie takich danych jak tytuł, opis, data rozpoczęcia i zakończenia oraz ustawienia powtarzalności (Rys. 13). Po zatwierdzeniu, dane są wysylane do serwera, a nowe wydarzenie jest natychmiast widoczne w kalendarzu i na osi czasu.

Formularz zadania pozwala na wprowadzenie tytułu, krótkiego opisu oraz priorytetu (Rys. 14). Po zatwierdzeniu, zadanie pojawia się w prawej kolumnie.



Rys. 15. Kalendarz miesięczny

Źródło: Opracowanie własne.



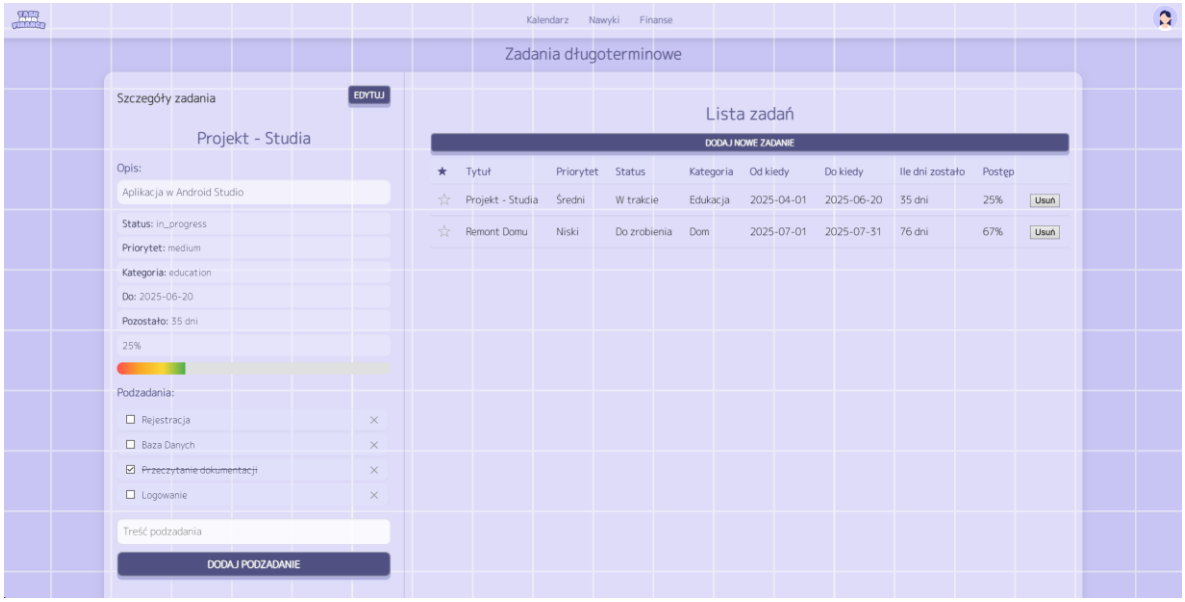
Rys. 16. Kalendarz tygodniowy

Źródło: Opracowanie własne.

Poza szczegółowym widokiem jednego dnia, aplikacja umożliwia także planowanie w szerszej perspektywie czasowej dzięki dwóm trybom wyświetlania kalendarza: miesięcznemu (Rys. 15) i tygodniowemu (Rys. 16). Przełączanie się pomiędzy nimi odbywa się za pomocą intuicyjnego przycisku w interfejsie.

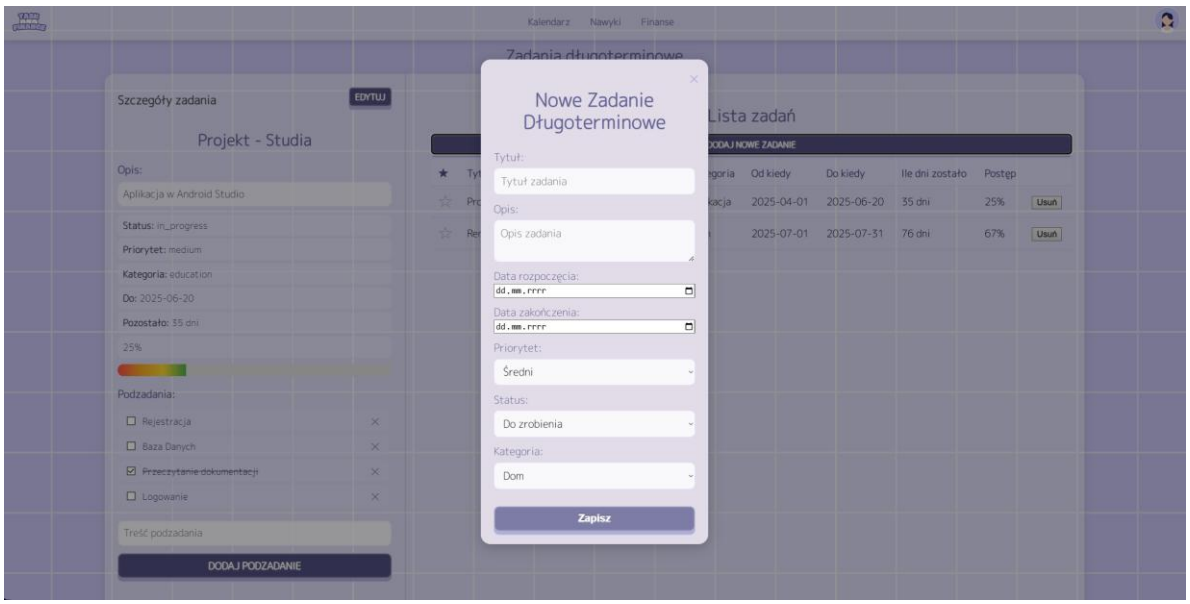
W trybie miesięcznym użytkownik widzi całą siatkę dni wybranego miesiąca. W każdym polu odpowiadającym konkretnemu dniowi wyświetlane są najważniejsze informacje o zaplanowanych wydarzeniach. Dzięki temu możliwa jest szybka ocena natężenia zajęć w danym okresie oraz łatwe przejście do szczegółów dowolnego dnia.

Przełączenie na widok tygodniowy pozwala skupić się na dokładnym rozkładzie zadań i wydarzeń w wybranym tygodniu. W tym trybie każdy dzień prezentowany jest w podziale na godziny, dzięki czemu możemy zobaczyć siatkę tygodnia.



Rys. 17. Lista zadań długich

Źródło: Opracowanie własne.



Rys. 18. Formularz zadania długoterminowego

Źródło: Opracowanie własne.

W dolnej części strony znajdziemy widok do zarządzania zadaniami długoterminowymi (Rys. 17). Zadania te są prezentowane w formie tabeli, która zawiera kluczowe informacje o projektach.

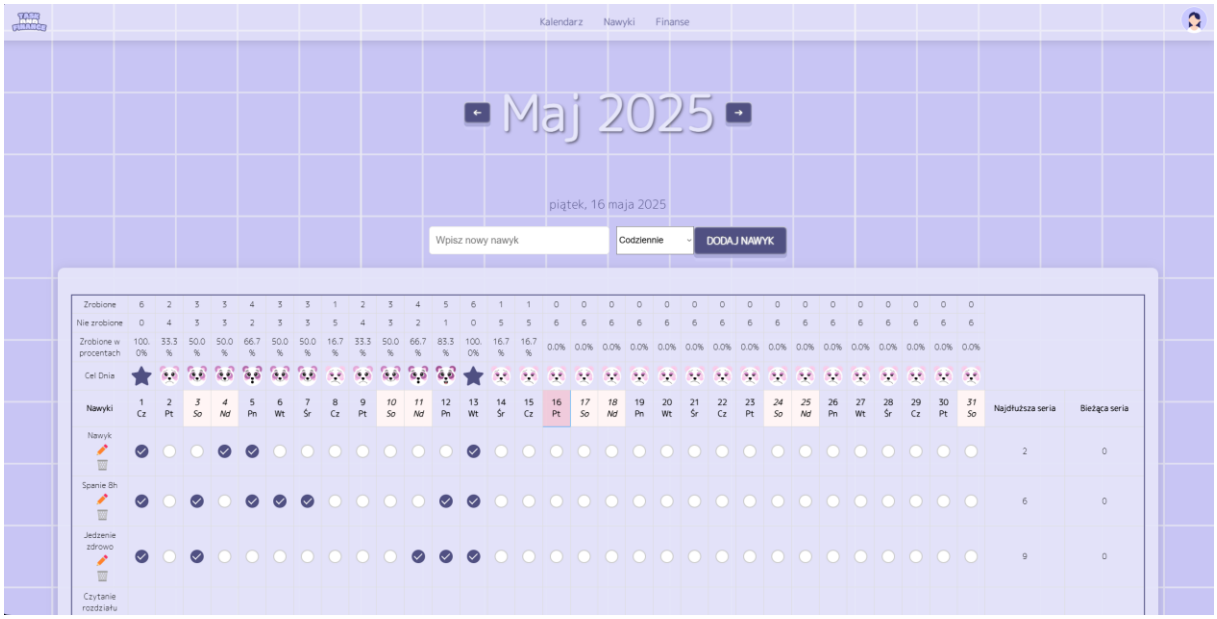
Po wybraniu zadania z listy, po lewej stronie pojawiają się szczegóły, w tym opis, priorytet, status, kategoria, data zakończenia i liczba dni pozostałych do terminu. Widok ten zawiera także graficzny pasek postępu oraz listę podzadań, które można odznaczać

jako wykonane lub usuwać. Dla każdego podglądu zadania wbudowany jest również formularz pozwalający na dodanie kolejnych podzadań.

Dodanie nowego zdania długoterminowego odbywa się przez modalny formularz, w którym użytkownik podaje podane wyżej parametry (Rys. 18). Po zapisaniu zadania automatycznie pojawia się w głównej tabeli.

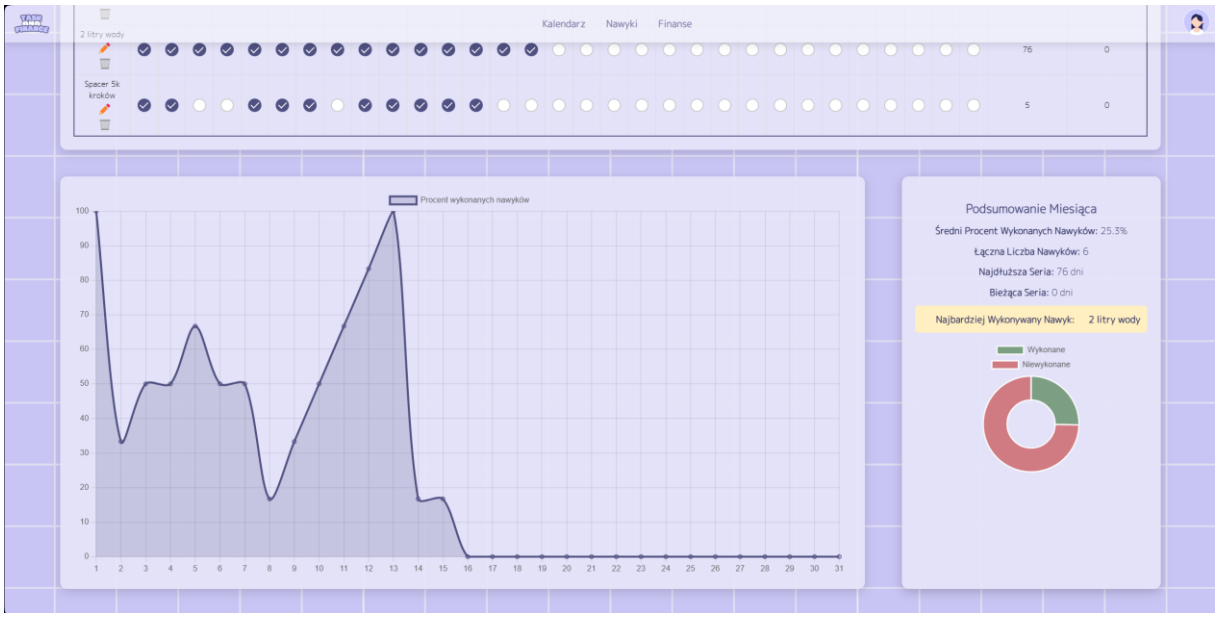
Wszystkie operacje na zadaniach i podzadaniach są obsługiwane po stronie frontendowej przez wywołania odpowiednich endpointów API. Procentowy postęp realizacji zadania wyliczany jest na podstawie liczby ukończonych podzadań względem ich całkowitej liczby.

III.4.5. Ekran nawyków



Rys. 19. Harmonogram nawyków

Źródło: Opracowanie własne.



Rys. 20. Wykres i statystyka dla nawyków

Źródło: Opracowanie własne.

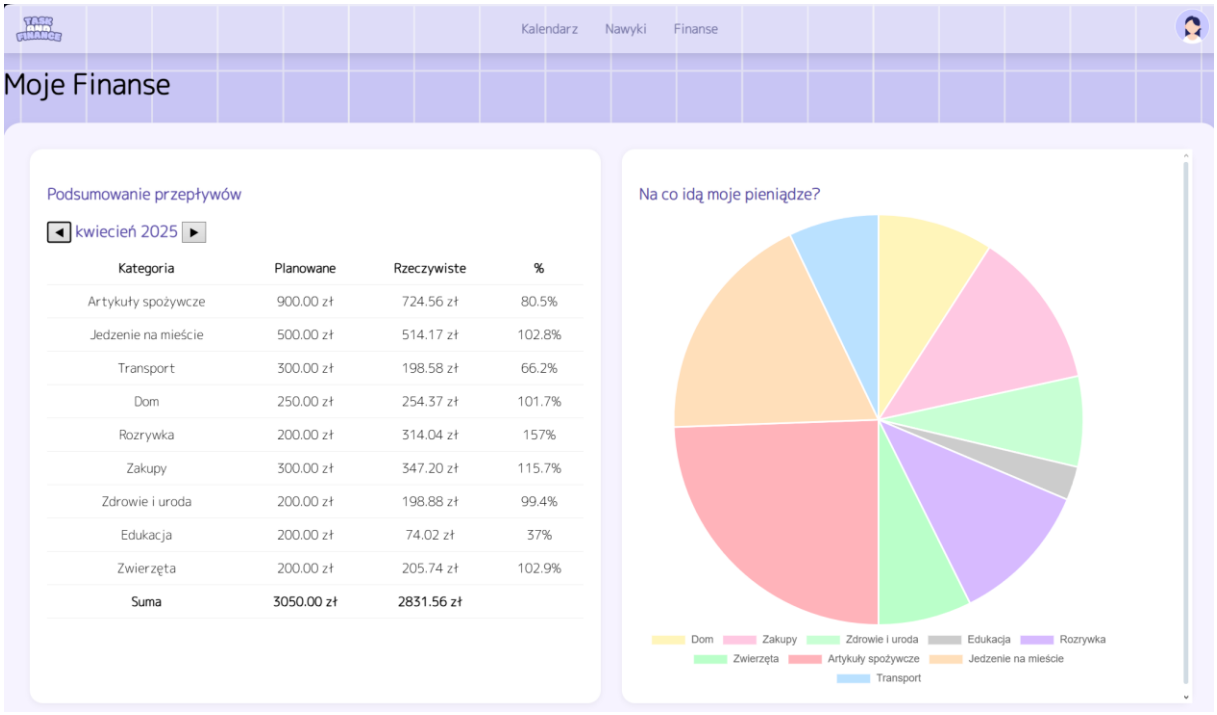
Kolejną sekcją dostępną z poziomu górnego menu aplikacji jest zakładka „Nawyki". Po przejściu do tej zakładki użytkownik zyskuje możliwość systematycznego monitorowania codziennych nawyków oraz analizowania ich realizacji.

Centralnym elementem widoku jest interaktywna tabela, w której każdemu nawykowi przypisany jest osobny wiersz, a każdemu dniowi miesiąca kolumna (Rys. 19). Użytkownik może w prosty sposób zaznaczać wykonanie danego nawyku w wybranym dniu, co natychmiast aktualizuje dane w systemie. Moduł automatycznie wylicza liczbę wykonanych oraz niewykonanych nawyków, procent realizacji i graficznie oznaczie „cel dnia" dla każdej daty.

U dołu sekcji znajduje się wykres liniowy prezentujący, jak zmieniał się poziom realizacji nawyków w kolejnych dniach miesiąca (Rys. 20). Dodatkowy panel statystyk po prawej stronie pokazuje średni wskaźnik realizacji, liczbę nawyków oraz długość aktualnej i najdłuższej serii. Wykres kołowy wizualizuje udział zrealizowanych i pominiętych nawyków, a najlepiej wykonywany nawyk jest wyróżniony.

Dodawanie nowego nawyku odbywa się za pomocą prostego formularza umieszczonego nad tabelą. Moduł nawyków pozwala użytkownikowi nie tylko kształtować dobre przyzwyczajenia, ale także szczegółowo analizować swoją regularność i postępy na przestrzeni wybranego miesiąca.

III.4.6. Ekran finansów



Rys. 21. Podsumowanie miesięczne wydatków

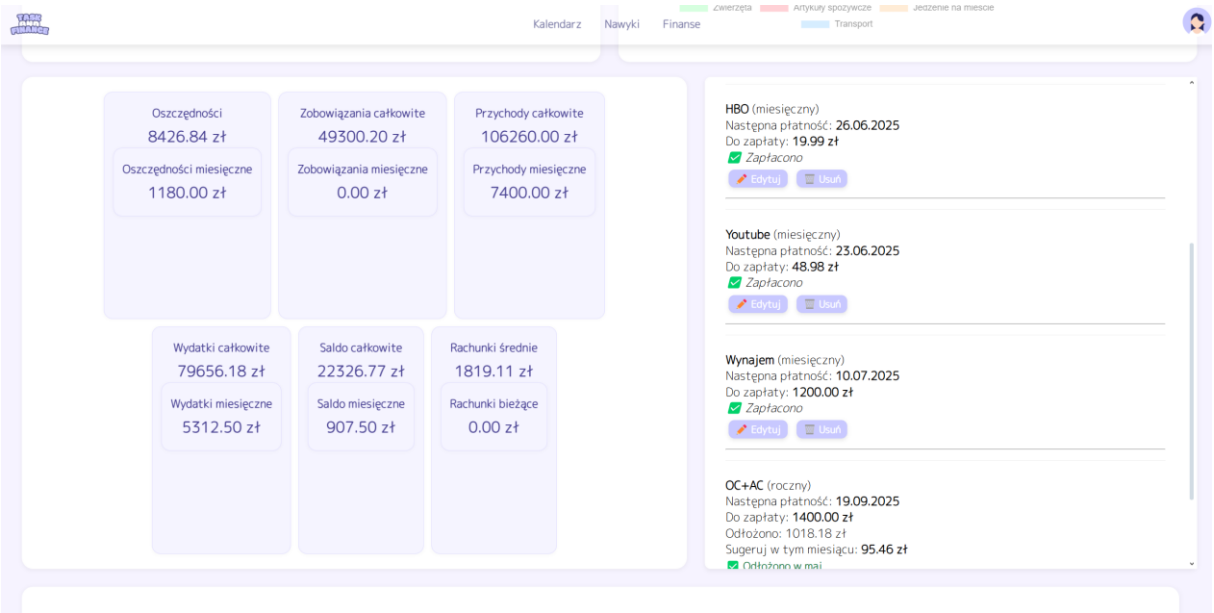
Źródło: Opracowanie własne.

Ostatnią kluczową zakładką w górnym menu aplikacji jest moduł „Finanse”. Po przejściu do tej sekcji użytkownik uzyskuje szczegółowy wgląd w swoje miesięczne przepływy finansowe, prezentowane w postaci podsumowań i wykresów (Rys. 21).

Centralnym elementem widoku jest tabela porównawcza wydatków według kategorii, podzielona na wartości planowane oraz rzeczywiste. Dla każdej kategorii wyświetlany jest także procent realizacji założonego budżetu, co pozwala użytkownikowi łatwo monitorować, w których obszarach przekracza ustalone limity. Przełączanie miesięcy umożliwiają przyciski nawigacyjne, a prezentowane dane pobierane są dynamicznie na podstawie wybranego okresu czasu.

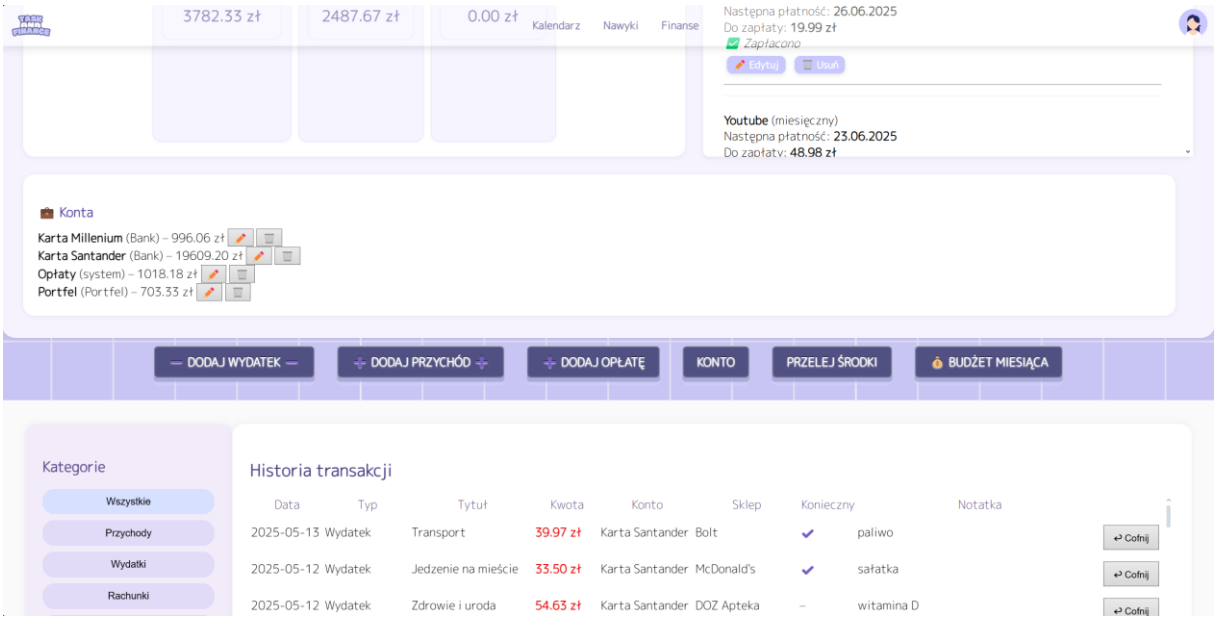
Obok zestawienia finansowego znajduje się wykres kołowy generowany za pomocą biblioteki Chart.js, który wizualizuje strukturę wydatków – użytkownik może szybko ocenić, na jakie kategorie przeznacza największą część swojego budżetu. Kolory poszczególnych sekcji wykresu odpowiadają kategoriom wydatków, co zapewnia czytelność i pozwala na błyskawiczną analizę proporcji.

Dzięki takiej konstrukcji interfejsu użytkownik może nie tylko kontrolować bieżące wydatki, ale również identyfikować tendencje i podejmować świadome decyzje finansowe w kolejnych miesiącach.



Rys. 22. Statystyki transakcji i lista rachunków

Źródło: Opracowanie własne.



Rys. 23. Widok kont użytkownika

Źródło: Opracowanie własne.

W kolejnej części strony użytkownik ma dostęp do panelu podsumowań, rachunków oraz narzędzi umożliwiających zarządzanie finansami (Rys. 22, Rys.23).

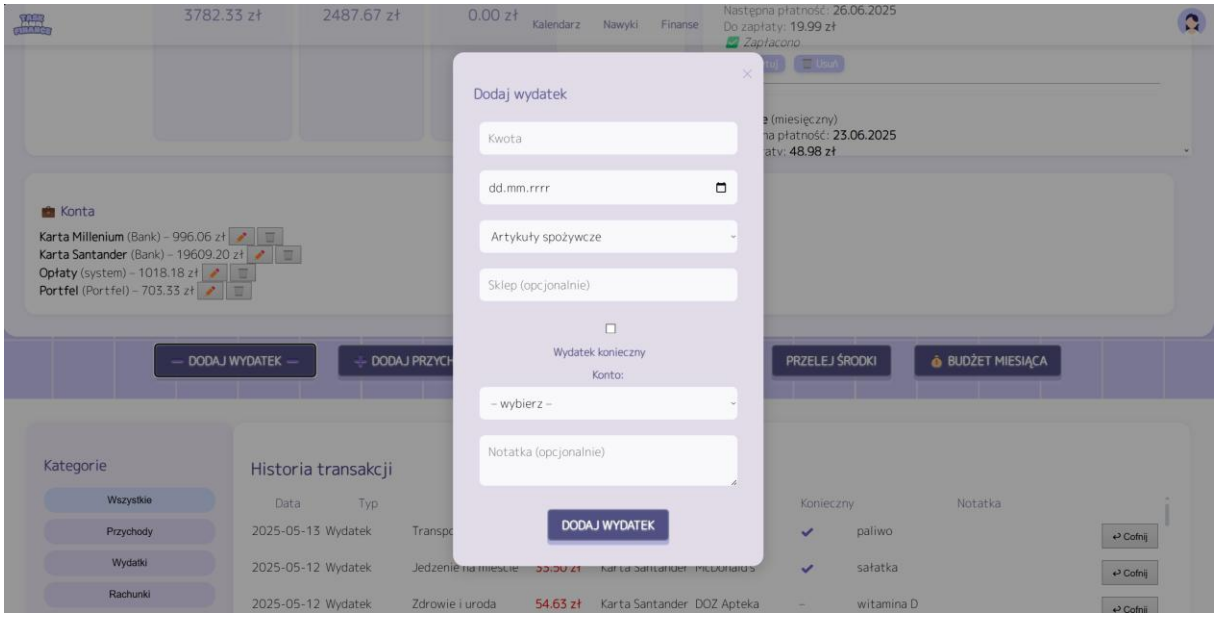
Po lewej stronie znajdują się kafelki statystyczne, prezentujące kluczowe agregaty finansowe. Po wykonaniu transkacji, dane dynamicznie pobierają dane z backendu i aktualizują widok.

Po prawej stronie znajduje się panel bieżących rachunków. Użytkownik widzi listę aktywnych opłat i subskrypcji, wraz z datą najbliższej płatności, kwotą oraz statusem. System wyraźnie wyróżnia, które rachunki zostały już opłacone, a które

wymagają uwagi. Dla każdego rachunku dostępne są przyciski do edycji lub usunięcia oraz informacje o kwocie sugerowanej do odłożenia w przypadku opłat rocznych.

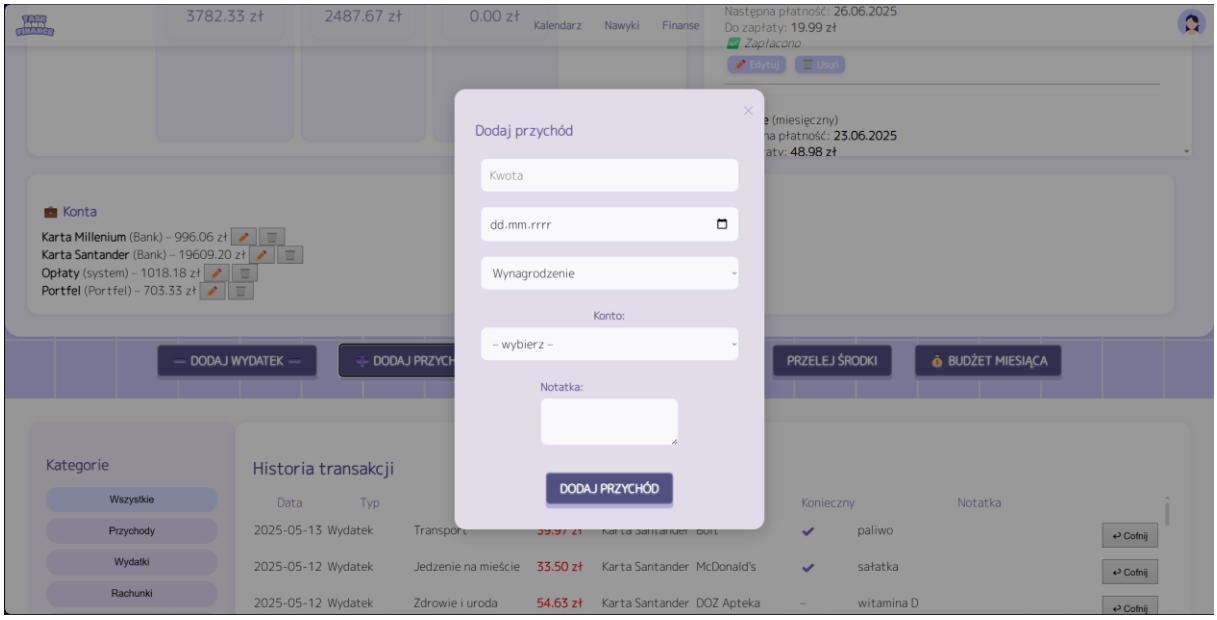
Następnie prezentowana jest lista wszystkich kont finansowych użytkownika, z rozbiciem na typ oraz z podaniem aktualnego salta. Każde konto można edytować i usuwać. Każdy użytkownik od początku ma tworzone konto „Opłaty” i „Portfel” automatycznie.

Bezpośrednio pod listą kont znajdują się szybkie przyciski umożliwiające dodanie nowego wydatku, przychodu, opłaty, konta, wykonanie przelewu lub zaplanowanie budżetu na wybrany miesiąc.



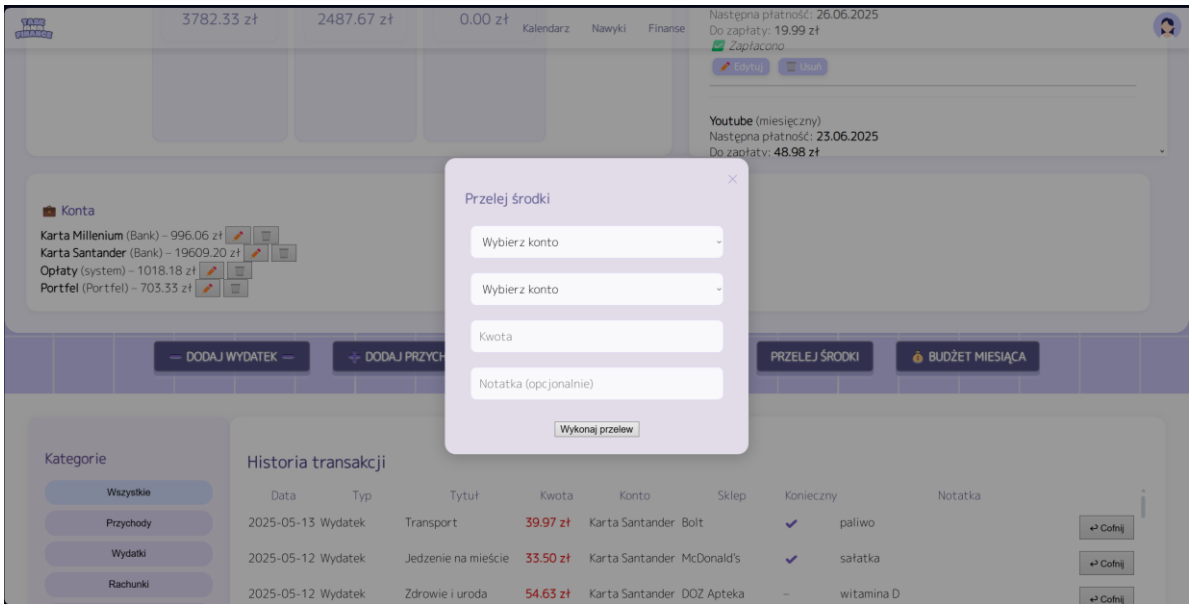
Rys. 24. Formularz wydatku

Źródło: Opracowanie własne.



Rys. 25. Formularz przychodu

Źródło: Opracowanie własne.



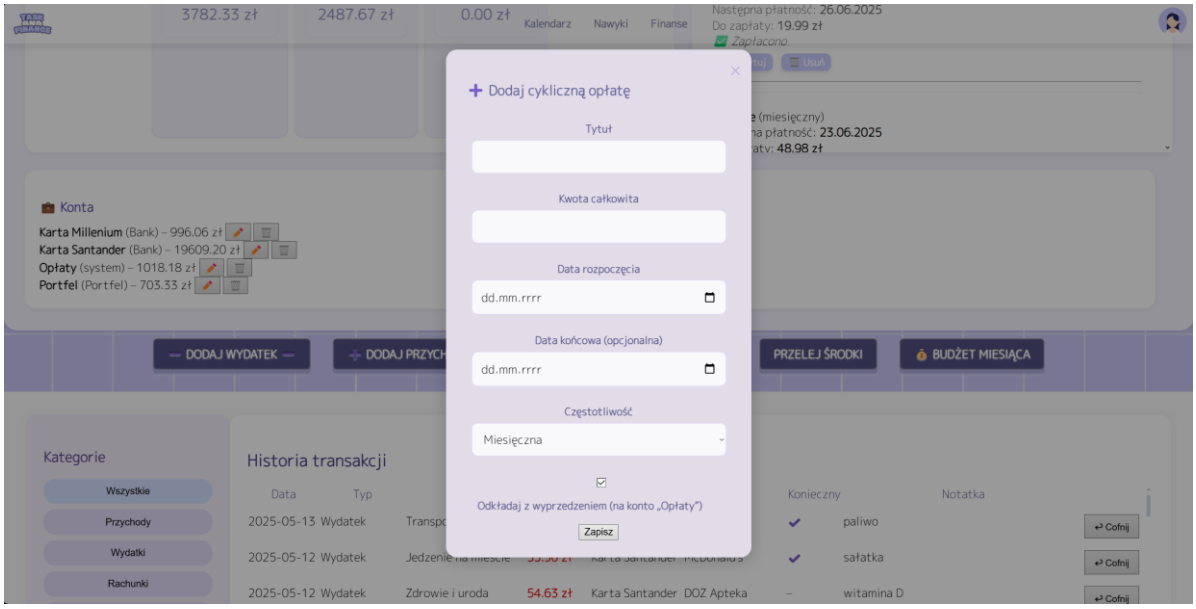
Rys. 26. Formularz przelewu środków

Źródło: Opracowanie własne.

Formularz dodawania wydatku otwierany jest w modalnym oknie po kliknięciu odpowiedniego przycisku w sekcji finansów (Rys. 24). Użytkownik wprowadza kwotę, datę, kategorię wydatku, opcjonalnie nazwę sklepu, oznacza, czy wydatek jest konieczny oraz wybiera konto, z którego środki mają zostać pobrane.

Analogiczny formularz służy do wprowadzania przychodu (Rys. 25). Użytkownik podaje kwotę, datę wpływu, źródło przychodu , wybiera konto docelowe oraz może opcjonalnie dodać notatkę do transakcji.

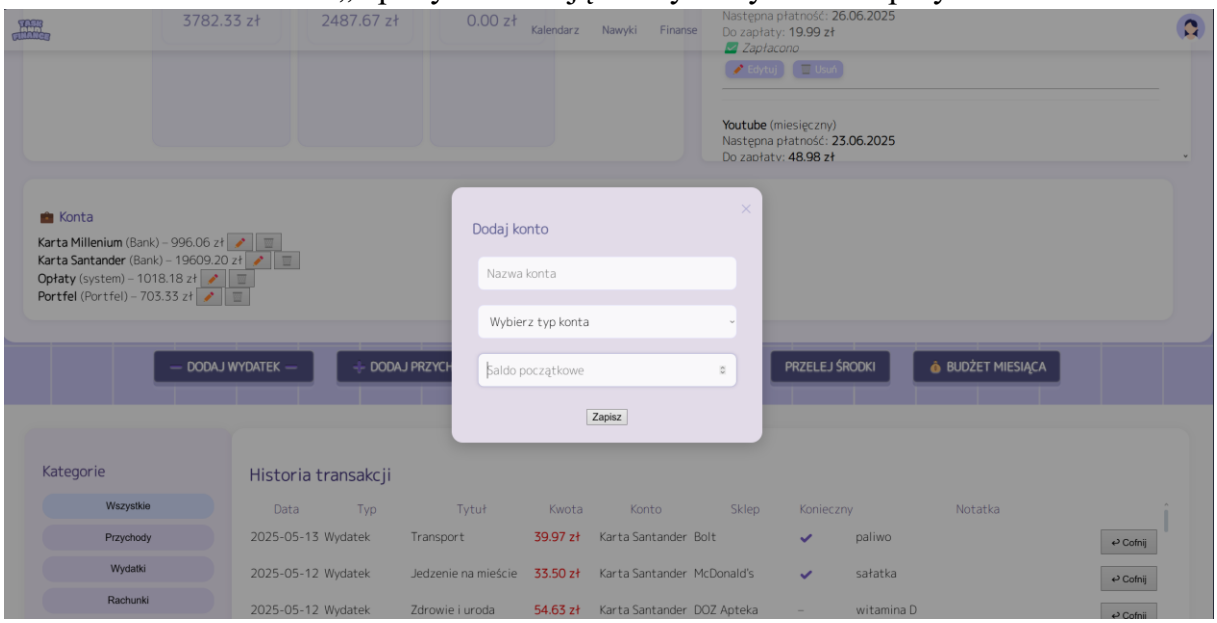
Formularz przelewu środków pozwala użytkownikowi na wykonanie wewnętrznego transferu pomiędzy własnymi kontami finansowymi (Rys. 26). Użytkownik wybiera konto źródłowe i docelowe z list, wpisuje kwotę przelewu oraz opcjonalnie notatkę opisującą operację.



Rys. 27. Formularz opłaty cyklicznej

Źródło: Opracowanie własne.

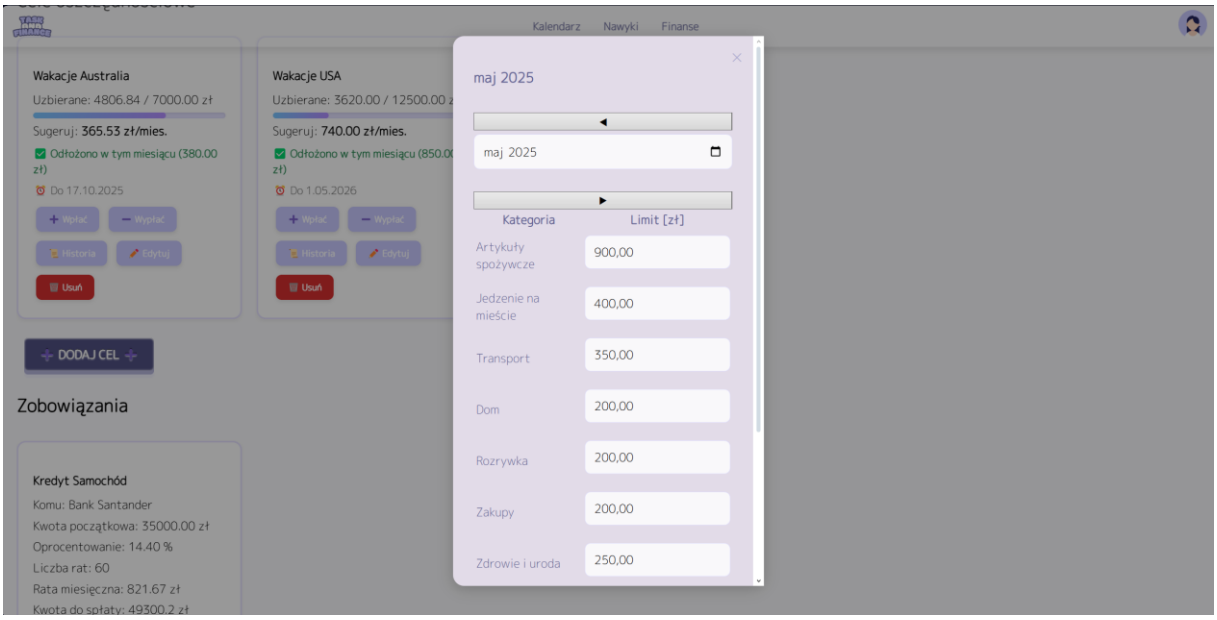
Formularz dodawania cyklicznej opłaty umożliwia zdefiniowanie nowego rachunku powtarzającego się w zadanym okresie (Rys. 27). Użytkownik wprowadza tytuł opłaty, całkowitą kwotę do zapłaty, datę rozpoczęcia oraz opcjonalnie datę końcową obowiązywania. Dostępna jest lista wyboru częstotliwości płatności (np. miesięczna, roczna). Dodatkowo, opcja „Odkładaj z wyprzedzeniem” pozwala automatycznie rozkładać wymagane środki na wyznaczone konto finansowe, umożliwiając systematyczne oszczędzanie na przyszłe rachunki. Środki są wtedy transferowane na konto „Opłaty” i czekają na wykorzystanie w przyszłości.



Rys. 28. Formularz dodania konta

Źródło: Opracowanie własne.

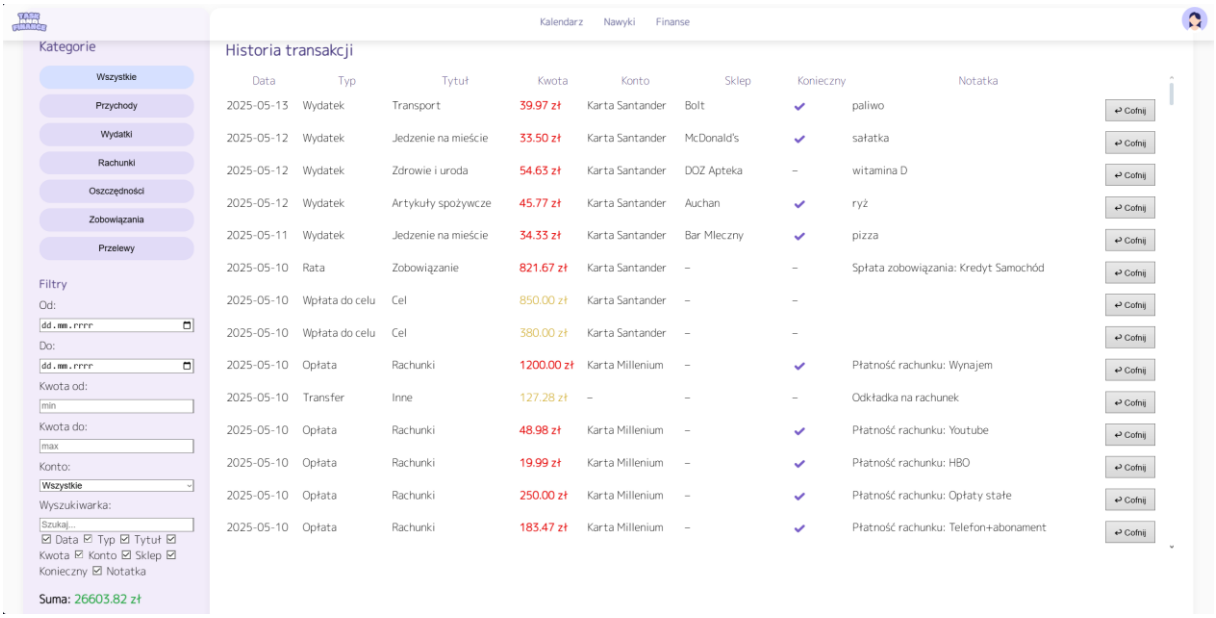
Formularz dodawania konta umożliwia użytkownikowi utworzenie nowego rachunku finansowego w systemie (Rys. 28). Użytkownik podaje nazwę konta, wybiera jego typ oraz ustala saldo początkowe. Saldo konta jest dynamicznie uwzględniane we wszystkich podsumowaniach i raportach finansowych aplikacji.



Rys. 29. Formularz ustawień budżetu miesięcznego

Źródło: Opracowanie własne.

W celu zarządzania wydatkami użytkownik ma do dyspozycji formularz budżetu miesięcznego, dostępny w postaci modalnego okna (Rys. 29). Po jego otwarciu użytkownik wybiera interesujący go miesiąc, dla którego chce ustalić limity wydatków. Formularz prezentuje przejrzystą tabelę, w której każda kategoria posiada własne pole do wprowadzenia planowanego limitu kwotowego na dany miesiąc. Wprowadzanie lub modyfikacja limitów odbywa się bezpośrednio w odpowiednich polach tabeli.



Rys. 30. Lista transakcji z możliwością filtracji

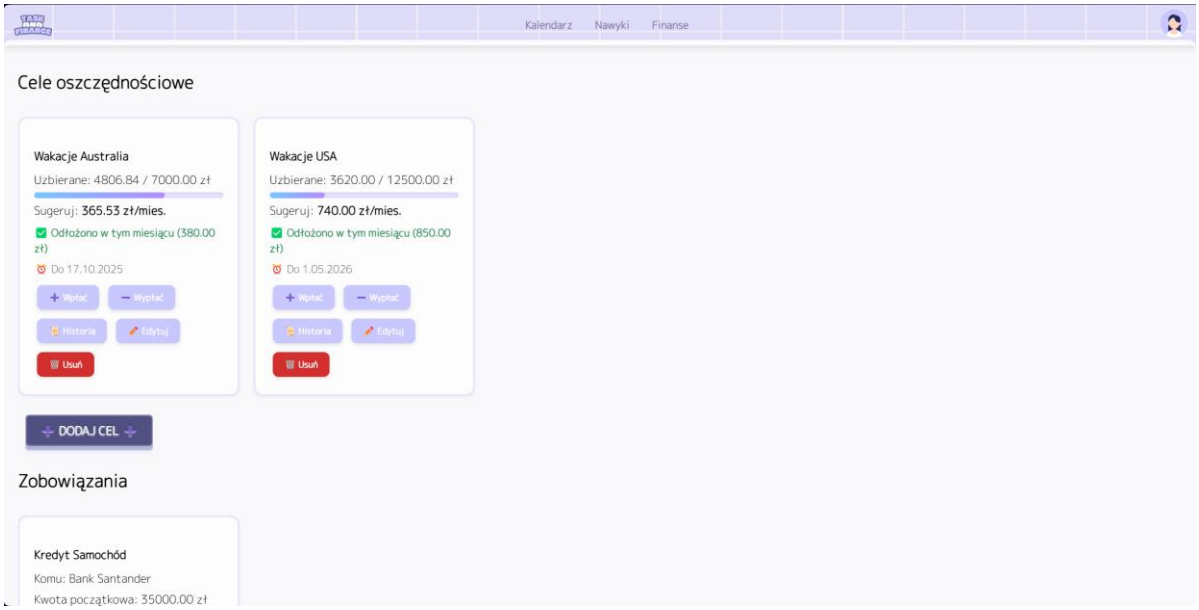
Źródło: Opracowanie własne.

W środkowej części sekcji finansowej znajduje się zaawansowany widok historii transakcji, umożliwiający pełną analizę wszystkich operacji finansowych użytkownika (Rys. 30). Po lewej stronie umieszczone są kategorie umożliwiające szybkie filtrowanie

operacji według typu. Dodatkowe filtry pozwalają zawęzić wyniki według zakresu dat, kwoty, wybranego konta lub sklepu, a także wyszukać operacje po frazie zawartej w tytule lub notatce. Centralnym elementem widoku jest tabela prezentująca szczegółowe informacje o każdej transakcji z kolorystycznym wyróżnieniem typów transakcji. Każdy rekord wyposażony jest w przycisk umożliwiający cofnięcie operacji, co pozwala na łatwą korektę błędnie wprowadzonych danych lub anulowanie przypadkowych operacji.

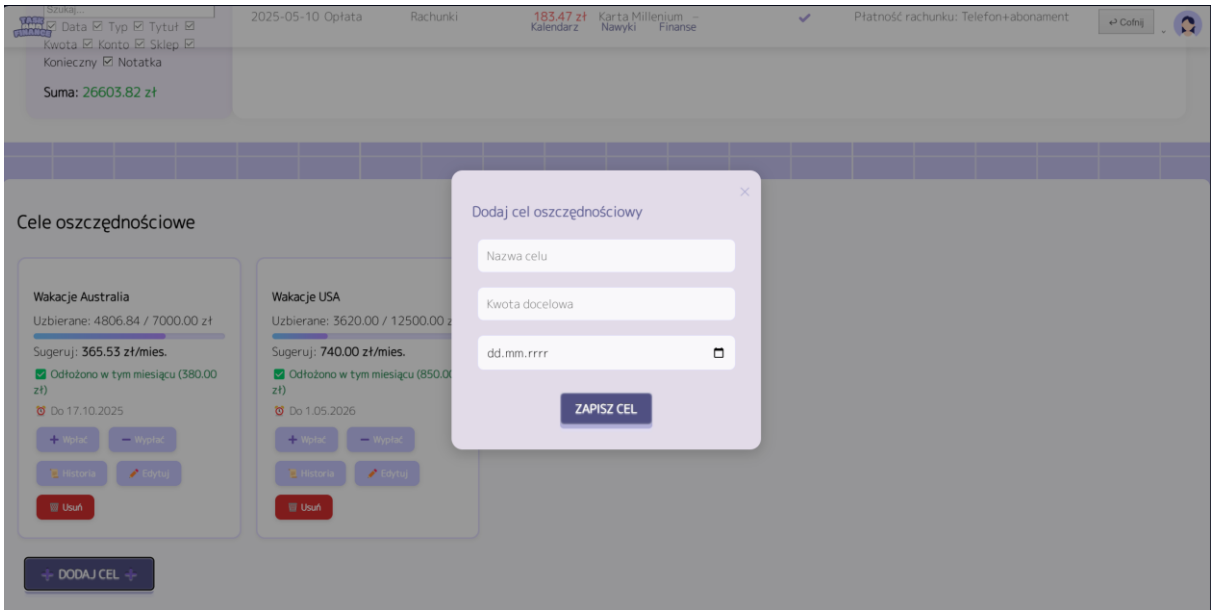
Na dole widoku wyświetlana jest suma aktualnie przefiltrowanych transakcji, automatycznie aktualizowana po każdej zmianie kryteriów wyszukiwania.

Widok jest dynamicznie synchronizowany z backendem, każda zmiana filtrów, cofnięcie lub dodanie operacji natychmiast odświeża prezentowaną listę i podsumowanie.



Rys. 31. Lista celów oszczędnościowych

Źródło: Opracowanie własne.

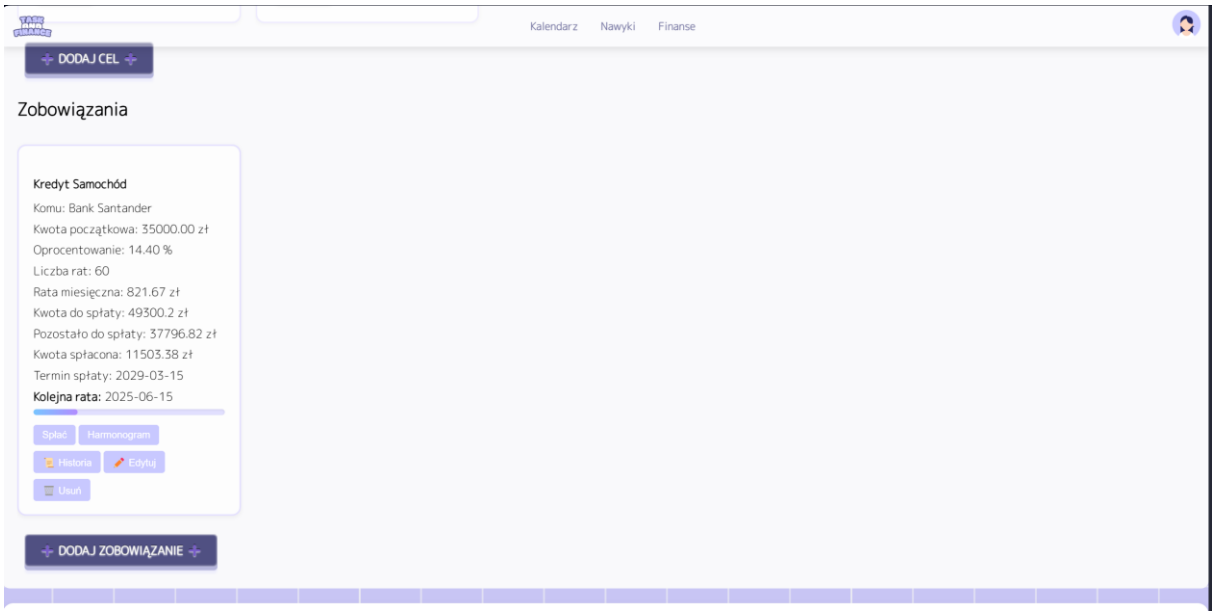


Rys. 32. Formularz celu

Źródło: Opracowanie własne.

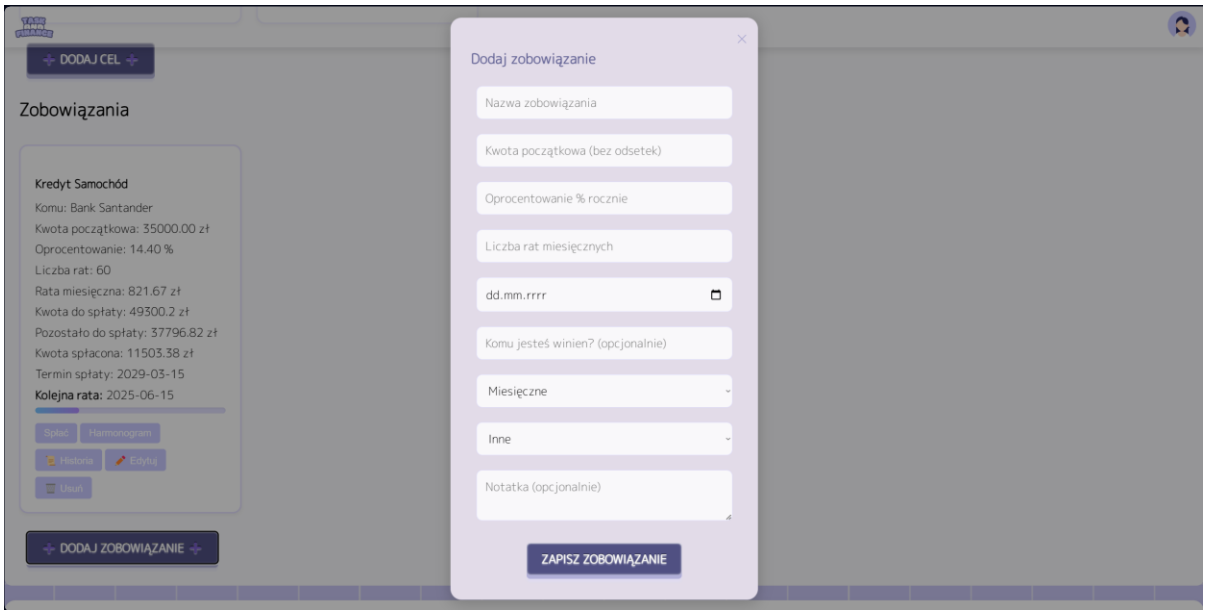
Moduł celów oszczędnościowych umożliwia użytkownikowi planowanie i monitorowanie postępu w realizacji własnych założeń finansowych, takich jak odkładanie na wyjazd czy duży zakup (Rys. 31). Każdy cel reprezentowany jest jako oddzielny kafelek prezentujący nazwę, kwotę zebraną, kwotę docelową, sugerowaną miesięczną wpłatę oraz termin realizacji. System dynamicznie wylicza rekomendowaną miesięczną kwotę do odłożenia oraz wskazuje, ile już odłożono w bieżącym miesiącu.

Do każdego celu przypisane są przyciski pozwalające na szybkie dodanie wpłaty lub wypłaty, podejrzenie historii operacji powiązanych z celem oraz edycję lub usunięcie celu. Dodawanie nowego celu odbywa się przez modalny formularz (Rys. 32). Użytkownik podaje nazwę celu, kwotę docelową oraz opcjonalnie termin realizacji.



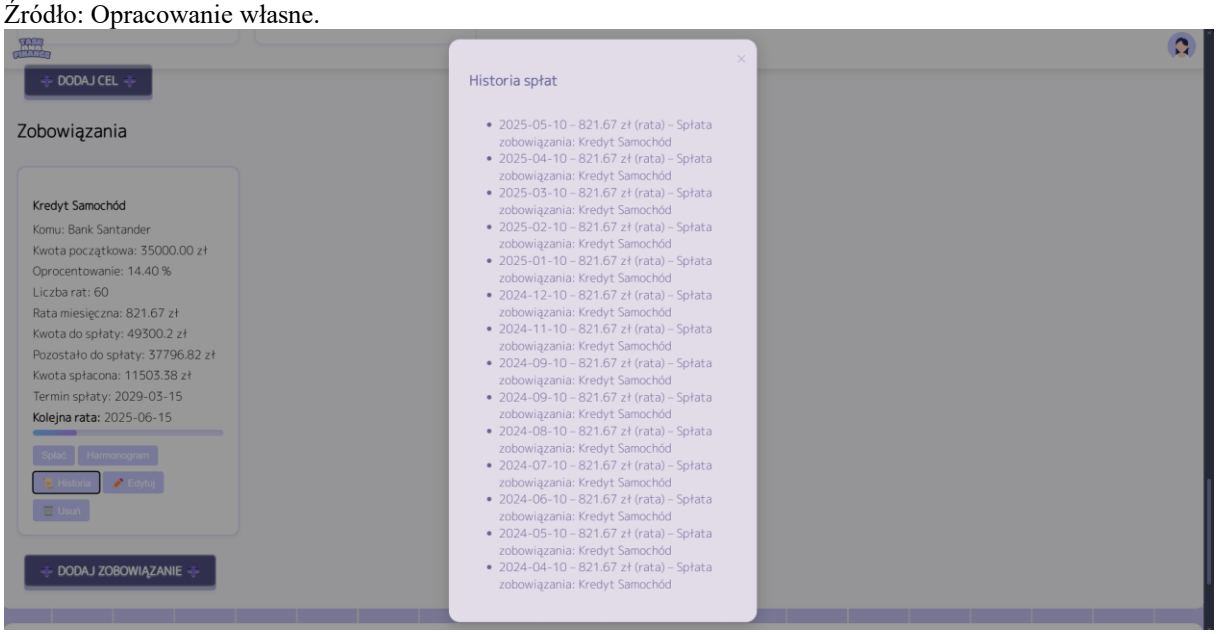
Rys. 33. Lista zobowiązań

Źródło: Opracowanie własne.



Rys. 34. Formularz zobowiązania

Źródło: Opracowanie własne.



Rys. 35. Historia transakcji zobowiązania

Źródło: Opracowanie własne.

Moduł zobowiązań umożliwia użytkownikowi ewidencjonowanie i śledzenie wszystkich długoterminowych zobowiązań finansowych, takich jak kredyty, pożyczki czy inne formy zadłużenia (Rys. 33). Każde zobowiązanie prezentowane jest w formie oddzielnego kafelka, w którym wyświetlane są szczegółowe dane. Przy każdym zobowiązaniu dostępne są przyciski umożliwiające dokonanie spłaty, wyświetlenie harmonogramu rat (Rys. 35), podejrzenie historii spłat, edycję oraz usunięcie zobowiązania.

Dodawanie nowego zobowiązania odbywa się przez modalny formularz, gdzie użytkownik wprowadza nazwę zobowiązania, kwotę początkową (bez odsetek), roczną

stopę oprocentowania, liczbę miesięcznych rat, datę rozpoczęcia, dane wierzyciela, częstotliwość rat oraz kategorię zobowiązania (Rys. 34).

Wysokość raty oraz całkowity koszt zobowiązania wyliczane są na podstawie zadanej przez użytkownika rocznej stopy oprocentowania. Użytkownik ma możliwość skorzystania z algorytmu opierającego się na typowej dla rynku finansowego metodzie rat annuitetowych (równych), szeroko stosowanej zarówno przez banki, jak i instytucje kredytowe ze względu na swoją przejrzystość i przewidywalność dla klienta [15].



Rys. 36. Wykres danych historycznych i predykcji

Źródło: Opracowanie własne.

Moduł prognoz finansowych umożliwia użytkownikowi analizę trendów przychodów i wydatków na przestrzeni minionych miesięcy oraz generowanie prognozy na przyszłość. Centralnym elementem widoku jest wykres słupkowy, na którym prezentowane są dane historyczne i predykowane wartości dla kolejnych miesięcy (Rys.36).

Linia pionowa oznacza moment przejścia od danych rzeczywistych do prognozy, dzięki czemu użytkownik zyskuje jasny podział między okresem historycznym a prognozowanym. Dane do wykresu pobierane są asynchronicznie z backendu po kliknięciu przycisku "Przelicz prognozę", który inicjuje wywołanie funkcji prognozującej

III.5. Opis wybranych implementacji

III.5.1. Logowanie

Proces logowania użytkownika został oparty na mechanizmach uwierzytelniania oferowanych przez Django. Całość rozpoczyna się na etapie frontendowym, gdzie

użytkownik wprowadza swój login oraz hasło do formularza. Po zatwierdzeniu formularza dane te są wysyłane do serwera jako żądanie POST na dedykowany logowaniu endpoint API. Po stronie backendu adres ten jest obsługiwany dzięki zapisowi w pliku „urls.py", gdzie adres został powiązany z widokiem obsługującym logowanie „path('api/login/', views.login_view, name='login')"

Tym samym każde zapytanie dotyczące logowania trafia bezpośrednio do funkcji „login_view" zdefiniowanej w pliku „views.py". To właśnie tutaj działa logika logowania.

Kod. 1. Fragment kodu logowania w views.py

```
1
2     @api_view(['POST'])
3     def login_view(request):
4         username = request.data.get('username')
5         password = request.data.get('password')
6
7         user = authenticate(request, username=username,
8     password=password)
9     if user is not None:
10        login(request, user)
11    else:
12        messages.error(request, 'Nieprawidłowy login lub hasło.')
13
14
```

Źródło: Opracowanie własne.

Pierwszą linią w kodzie (Kod. 1) po dekoratorze widoku i definicji funkcji jest pobranie wartości loginu, a następnie hasła, przekazanego z frontendu. Dzieje się to za pomocą żądania „request.data" z metodą „get", która wyciąga wartości przypisane do kluczy „username" i „password". Po wydostaniu danych, aplikacja korzysta z funkcji „authenticate", która jest jednym z elementów wbudowanego mechanizmu Django. Podany login i hasło są sprawdzane względem bazy danych. Django wykonuje to w bezpieczny sposób: nie operuje na jawnych hasłach, lecz porównuje ich skróty (hash). Jeśli wszystko się zgadza, „authenticate" zwraca obiekt użytkownika. Jeśli nie, zwraca „None".

Kolejnym krokiem jest sprawdzenie, czy uwierzytelnienie się powiodło, co jest robione poprzez instrukcję „if user is not None:". Jeśli użytkownik został poprawnie zweryfikowany, uruchamiana jest funkcja „login(request, user)". To polecenie powoduje założenie sesji użytkownika. Django tworzy specjalny wpis w swojej bazie sesji oraz wysyła przeglądarce unikalny identyfikator sesji zalogowanego użytkownika. Dzięki temu przy każdym kolejnym żądaniu do aplikacji backend będzie mógł rozpoznać, który użytkownik jest aktywny.

Na końcu funkcji wysyłana jest odpowiedź do frontendu. W przypadku udanego logowania zwracany jest JSON zawierający komunikat sukcesu, natomiast przy niepoprawnych danych funkcja zwracana komunikat o błędnych danych.

III.5.2. Dodawanie konta finansowego

Cały proces rozpoczyna się po stronie frontendowej, gdzie użytkownik uzupełnia formularz służący do założenia nowego konta finansowego. Wprowadza tam nazwę konta, wybiera jego typ oraz ustawia saldo początkowe. Po zatwierdzeniu formularza, dane są przesyłane w formacie JSON jako żądanie POST na endpoint. Na backendzie obsługa tego żądania zaczyna się w pliku „urls.py", gdzie odpowiednia ścieżka została powiązana z funkcją obsługującą dodawanie kont „path("api/accounts/", views.get_or_create_accounts, name="accounts")". Dzięki tej konfiguracji każde żądanie dotyczące utworzenia nowego konta trafia bezpośrednio do widoku „get_or_create_accounts" zdefiniowanego w pliku „views.py".

Kod. 2. Fragment kodu tworzenia konta finansowego w views.py

```
1     @api_view(["GET", "POST"])
2     @permission_classes([IsAuthenticated])
3     def get_or_create_accounts(request):
4       user = request.user
5
6       … # kod niedotyczący dodawania konta finansowego
7
8       elif request.method == "POST":
9          serializer = AccountSerializer(data=request.data,
10         context={'request': request})
11         if serializer.is_valid():
12            serializer.save(user=user)
13            return Response(serializer.data,
14                            status=status.HTTP_201_CREATED)
15         return Response(serializer.errors,
16                         status=status.HTTP_400_BAD_REQUEST)
```

Źródło: Opracowanie własne.

Na początku fragmentu kodu (Kod. 2) nad funkcją widoku znajdują się dwa dekoratory „@api_view(["GET", "POST"])" informuje Django REST Framework, że funkcja obsługuje żądania typu GET oraz POST, natomiast „@permission_classes([IsAuthenticated])" wymusza, by użytkownik był zalogowany, aby móc korzystać z tej funkcji.

Pierwszą operacją po dekoratorach widoku oraz definicji funkcji jest pobranie aktualnie zalogowanego użytkownika, co realizowane jest przez odczyt atrybutu „request.user". Pozwala to jednoznacznie przypisać wszystkie kolejne operacje do użytkownika, który wysłał żądanie do backendu.

Następnie, jeśli żądanie otrzymane przez endpoint ma metodę "POST", rozpoczyna się proces obsługi dodawania nowego konta. Dane przesłane przez frontend w ciele żądania są przekazywane do serializera „AccountSerializer”. Dodatkowo jest używany „context={'request': request})”, co może być wykorzystywane przez serializer do walidacji zależnej od użytkownika lub innych parametrów.

Kolejnym krokiem jest sprawdzenie poprawności przesłanych danych za pomocą metody „is_valid()”. Jeśli walidacja się powiedzie, serializer zapisuje nowy obiekt konta w bazie danych, automatycznie przypisując je do zalogowanego użytkownika „serializer.save(user=user)”. Po poprawnym utworzeniu konta, aplikacja odsyła do frontendu odpowiedź z dodanymi danymi oraz kodem statusu HTTP 201, co potwierdza, że konto zostało utworzone.

Kod. 3. Fragment kodu tworzenia konta finansowego w serializers.py

```
class AccountSerializer(serializers.ModelSerializer):
  account_type_label = serializers.CharField(
      source='get_account_type_display', read_only=True)

  class Meta:
      model  = Account
      fields = ['id', 'name', 'account_type',
             'account_type_label', 'balance', 'is_archived']
      extra_kwargs = {'is_archived': {'read_only': True}}

  def create(self, validated_data):
      opening_balance=
             Decimal(validated_data.pop("balance", 0) or 0)
      user = self.context["request"].user
      account=
         Account.objects.create(user=user, **validated_data)

      if opening_balance > 0:
         Transaction.objects.create(
             user = user,
             amount = opening_balance,
             date = date.today(),
             tran_type = "income",
             source = "opening_balance",
             note= f"Saldo początkowe konta “{account.name}”",
             account = account,
             category = "other",
            )
         account.balance = opening_balance
         account.save(update_fields=["balance"])

      return account
```

Źródło: Opracowanie własne.

Przetwarzanie danych dotyczących nowego konta finansowego po stronie backendu realizowane jest za pomocą klasy AccountSerializer w fragmencie przedstawionego kodu (Kod. 3). Na początku w serializerze zdefiniowane zostaje dodatkowe pole „account_type_label”, które korzystaz wbudowanej metody modelu Django „get_account_type_display”, pozwalając na wyświetlanie czytelnej, opisowej nazwy typu konta w odpowiedzi zwracanej do frontendu. Pole to jest oznaczone jako tylko do odczytu.

W sekcji konfiguracyjnej „Meta” określono, że serializer pracuje na modelu „Account”, a do odpowiedzi trafiają wyłącznie wybrane pola. Dodatkowo pole „is_archived” zostało ustawione jako tylko do odczytu, aby uniemożliwić jego zmianę podczas tworzenia konta.

W dalszej części serializer nadpisuje metodę „create”, która odpowiada za utworzenie nowego konta oraz ewentualne zainicjowanie transakcji, jeśli saldo początkowe jest większe od zera. W pierwszym kroku metoda ta pobiera saldo początkowe „balance” z przekazanych danych, jeśli użytkownik nic nie wpisał, wtedy wynosi 0. Następnie z kontekstu serializera pobierany jest obiekt aktualnie zalogowanego użytkownika, co pozwala na powiązanie nowego konta z konkretną osobą w systemie. Kolejnym krokiem jest utworzenie nowego obiektu „Account” w bazie danych. Jeżeli saldo początkowe jest większe od zera, aplikacja automatycznie generuje pierwszą transakcję powiązaną z danym kontem, jest to transakcja typu „przychód”, której źródłem jest „saldo początkowe”, a w opisie umieszczana jest informacja o otwarciu rachunku. Równocześnie saldo konta jest aktualizowane do zadanej wartości i zapisywane w bazie. W rezultacie metoda zwraca nowo utworzony obiekt konta, który następnie trafia w odpowiedzi zwrotnej do użytkownika frontendowego.

Model konta zdefiniowany jest w pliku „models.py” i zawiera podstawowe informacje o koncie oraz powiązanie z użytkownikiem.

Kod. 4. Fragment kodu klasy tworzenia konta finansowego w models.py

```
1     Class Account(models.Model):
2         ACCOUNT_TYPES = [
3             ("piggy",  "Skarbonka"),
4             ("bank",   "Bank"),
5             ("wallet", "Portfel"),
6             ("gift",   "Karta podarunkowa"),
7             ("other",  "Inne"),
8         ]
9         user = models.ForeignKey(User, on_delete=models.CASCADE)
10        name = models.CharField(max_length=100)
11        account_type = models.CharField(max_length=20,
12                        choices=ACCOUNT_TYPES)
13        balance= models.DecimalField(max_digits=12,
14                        decimal_places=2, default=0)
```

```
15        is_archived = models.BooleanField(default=False)
16        def __str__(self):
17            return f"{self.name}
18                        ({self.get_account_type_display()})"
```

Źródło: Opracowanie własne.

Dzięki tej strukturze, nowe konto jest powiązane z właścicielem oraz posiada wszystkie wymagane atrybuty, takie jak typ czy saldo. Na początku kodu (Kod. 4) w modelu znajduje się stała „ACCOUNT_TYPES”. To lista opcji, z która składa się z dwóch części, pierwsza to techniczna nazwa typu konta, a druga to opis, który zobaczy użytkownikw aplikacji.

Pole „user” to klucz obcy „ForeignKey” wskazujący na model użytkownika „User”. Każde konto jest więc przypisane do jednego użytkownika. Opcja „on_delete = models.CASCADE” sprawia, że jeśli użytkownik zostanie usunięty z systemu, wszystkie jego konta również zostaną usunięte automatycznie, dzięki mechanizmowi „kaskady” w relacjach bazy danych. Model również przechowuje nazwę konta „name”, wybrany typ konta „account_type”, aktualne saldo konta „balance” oraz informacje czy konto jest zarchiwizowane „is_archived”.

Podsumowując, pełny proces dodania konta obejmuje przesłanie danych przez frontend, przekazanie ich przez URL do odpowiedniego widoku, walidację i obsługę w serializerze, zapis do bazy w modelu „Account”. Odpowiedź trafia z powrotem do użytkownika w formacie JSON, co umożliwia dalszą, automatyczną obsługę po stronie interfejsu użytkownika.

III.5.3. Pozostałe implementacje

Wszystkie kluczowe moduły aplikacji obejmujące zarządzanie wydarzeniami kalendarza, nawykami, zadaniami, podzadaniami oraz innymi obiektami użytkownika zostały zaprojektowane zgodnie z jednolitą logiką REST API. Zapewniającą spójność i przejrzystość działania całego systemu. Oznacza to, że każda operacja na danych, niezależnie czy dotyczy tworzenia, pobierania, edycji czy usuwania, realizowana jest według tych samych zasad oraz przepływu kodu.

Dla każdego typu danych w aplikacji zdefiniowano osobne endpointy w pliku „urls.py”, kierujące żądania do właściwych widoków w pliku „views.py”. Widoki te obsługują najczęściej kilka metod http: „GET” (pobieranie danych), „POST” (tworzenie nowego obiektu), „PUT/PATCH” (edycja), „DELETE” (usuwanie). Każda z tych operacji przekazuje dane do dedykowanego serializera, który odpowiada za walidację i konwersję danych pomiędzy formatem JSON a obiektami Django. Odpowiednie uprawnienia „@permission_classes([IsAuthenticated])” dbają o to, by tylko zalogowani użytkownicy mogli zarządzać własnymi danymi.

W każdym przypadku obsługa CRUD (Create, Read, Update, Delete) odbywa się w analogiczny sposób. Nawet jeśli poszczególne modele mają specyficzne powiązania (np. podzadanie musi być powiązane z zadaniem nadrzędnym, „HabitDay” przypisany

do Habit), całość opiera się na tej samej architekturze. Odpowiedni serializer weryfikuje, czy takie powiązanie istnieje, a logika widoku zapewnia właściwą obsługę wszystkich operacji.

Chociaż wszystkie moduły aplikacji korzystają ze wspólnego schematu komunikacji, poszczególne modele różnią się między sobą szczegółami implementacji oraz charakterystyką przetwarzania danych.

- W przypadku zadań i podzadań logika wymaga zachowania powiązania hierarchicznego, każde podzadanie zawsze musi być przypisane do konkretnego zadania nadrzędnego, co wymusza dodatkową walidację w serializerze i widoku.
- Dla nawyków oraz wpisów „HabitDay" istotna jest ciągłość i historia wykonywania, model „HabitDay" istnieje wyłącznie jako część cyklu konkretnego nawyku i przechowuje informacje o jego realizacji w danym dniu.
- Obsługa wydarzeń w kalendarzu wiąże się z koniecznością przechowywania bardziej złożonych danych, takich jak zakresy czasowe, powtarzalność czy kolory zdarzeń, co wpływa na strukturę danych przesyłanych do API i ich walidację.
- W zadaniach i projektach pojawia się często mechanizm statusów, priorytetów oraz relacji z użytkownikiem.

Warto również zauważyć, że różnice mogą dotyczyć automatyzacji pewnych procesów.

III.6. Prezentacja modelu predykcji

Kluczowym elementem aplikacji finansowej jest moduł odpowiedzialny za generowanie prognoz przyszłych przychodów i wydatków. W ramach realizacji tego zadania przeprowadzono analizę dostępnych technik predykcyjnych i wstępne testy różnych modeli, w tym modeli opartych na regresji liniowej, modelach wygładzania wykładniczego (Exponential Smoothing) oraz Prophet (Facebook Prophet).

Pierwszym podejściem był model Prophet [16], który ze względu na swoją popularność i dostępność bibliotek wydawał się optymalnym rozwiązaniem. Prophet oferuje wbudowaną obsługę trendów, sezonowości i niestandardowych efektów okresowych, przez co teoretycznie powinien być odpowiedni dla szerokiego zakresu danych czasowych. W trakcie eksperymentów z jego implementacją napotkano jednak poważne ograniczenia wynikające przede wszystkim z niewielkiej ilości danych historycznych (tylko 12 pełnych miesięcy obserwacji). Jak wskazuje oficjalna dokumentacja Prophet, optymalne wyniki wymagają większej ilości danych. W tego typu aplikacji finansowej większość użytkowników nie miałaby wystarczającej liczby danych do przeprowadzenia poprawnej prognozy. W konsekwencji podjęto decyzję o rezygnacji z Prophet na rzecz klasycznych, mniej wymagających metod statystycznych.

Ostatecznie w module predykcji zastosowano metodę wygładzania wykładniczego z trendem addytywnym Exponential Smoothing [17], która okazała się optymalnym wyborem dla ograniczonej liczby obserwacji oraz występowania nieregularnych zmian w danych finansowych. Duża stabilność modelu nawet przy krótkich szeregach czasowych sprawia, że jest użyteczny tam, gdzie użytkownicy posiadają jedynie niewielką liczbę danych. Dodatkową zaletą wygładzania wykładniczego jest odporność na występowanie pojedynczych wartości odstających, które mogłyby zaburzyć parametry wyznaczanego trendu.

Aby jeszcze bardziej ograniczyć wpływ obserwacji nietypowych, przed rozpoczęciem prognozowania wprowadzono dodatkowy etap przetwarzania danych w postaci winsoryzacji kwot przychodów i wydatków na próg 5%, co oznacza, że skrajne wartości zarówno po stronie najmniejszych, jak i największych obserwacji zostały obcięte do granicznych wartości. Pozwoliło to ograniczyć ryzyko, że pojedyncze, nadzwyczajnie wysokie lub niskie wydatki wpłyną na kształt prognozowanej ścieżki finansowej użytkownika. Wprowadzenie winsoryzacji w połączeniu z wygładzaniem wykładniczym w istotny sposób podniosło jakość generowanych prognoz, zapewniając większą stabilność i wiarygodność przewidywań.

Wybrany model pozwala skutecznie uchwycić ogólny trend przychodów i wydatków użytkownika bez nadmiernej wrażliwości na krótkoterminowe wahania czy anomalie danych. Prognozy generowane w aplikacji są nie tylko matematycznie poprawne, ale też użyteczne i intuicyjne z punktu widzenia osoby zarządzającej własnym budżetem.

Kod. 5. Pobieranie danych źródłowych z bazy

```
today = date.today()
this_month_start = today.replace(day=1)
start_12_months = this_month_start - relativedelta(months=12)

qs = (Transaction.objects
      .filter(user=user,date__gte=start_12_months,
              date__lt=this_month_start)
      .exclude(tran_type="transfer")
      .values("date", "tran_type", "amount"))
if not qs:
    return [], None, None
```

Źródło: Opracowanie własne.

Pierwszym krokiem w procesie generowania prognozy finansowej jest precyzyjne określenie zakresu analizowanych danych historycznych. W tym celu wykorzystywane są operacje na datach oraz mechanizmy Django. Bieżący miesiąc nie jest uwzględniany, ponieważ posiada niepełne transakcje w danym miesiącu. Dane te są filtrowane, pobierane są tylko transakcje typu przychód "income" oraz wydatki "expense", "bill_payment", "liab_payment", "goal_deposit". Transakcje typu „transfer” (transfery

między kontami) są wykluczane, by uniknąć zafałszowania rzeczywistych przepływów finansowych (Kod. 5).

Na początku do zmiennej „today” przepisana zostaje aktualna data. Dzięki temu możemy następnie wyznaczyć pierwszy dzień bieżącego miesiąca za pomocą „today.replace(day=1)” oraz wyznaczyć również datę początku analizy przesuwając się dokładnie o 12 miesięcy wstecz od początku bieżącego miesiąca „this_month_start - relativedelta(months=12)”.

Fragment kodu „Transaction.objects.filter()” służy do pobierania transakcji należących do bieżącego użytkownika „user=user”, oraz w określonym zakresie dat wyznaczonych wcześniej. Używana jest do tego metoda „filter()”. Dodatkowo, wykluczane są wszystkie transakcje typu „transfer” za pomocą „exclude”, ponieważ dotyczą one przesunięć środków między własnymi kontami użytkownika i nie powinny wpływać na obraz rzeczywistych przychodów i wydatków. Ostatecznie, metoda „.values("date", "tran_type", "amount")” sprawia, że z każdego rekordu pobierane są jedynie kluczowe informacje takie jak data operacji, jej typ oraz kwota, co upraszcza dalszą analizę i umożliwia agregację danych. Po pobraniu danych, następuje sprawdzenie, czy pobrane zostały jakiekolwiek dane. W przypadku braku zwracana jest pusta prognoza i historia.

Kod. 6. Agregacja miesięczna danych

```
    df = pd.DataFrame.from_records(qs)
    df["date"] = pd.to_datetime(df["date"])
    df["amount"] = df["amount"].astype(float)

    df["cat"] = df["tran_type"].apply(
        lambda k: "income" if k == "income" else ("expense" if k
        in EXP_ALIAS else "ignore"))
    df = df[df["cat"] != "ignore"]

    df["month"] = df["date"].dt.to_period("M").dt.to_timestamp()
    monthly = (df.groupby(["month", "cat"])["amount"]
                 .sum()
                 .unstack(fill_value=0)
                 .sort_index())
```

Źródło: Opracowanie własne.

Uzyskane dane konwertowane są do formatu „DataFrame” z biblioteki pandas, a następnie agregowane miesięcznie z podziałem na dwie podstawowe kategorie: "income" oraz "expense". W efekcie powstaje macierz danych, w której każdy miesiąc posiada osobną sumaryczną wartość przychodów i wydatków użytkownika (Kod. 6).

Rekordy transakcji „qs” są zamieniane na obiekty typu DataFrame i przypisywane do zmiennej „df”.

Następnie konwertujemy kolumnę „date” do formatu „datetime”, aby umożliwić dalsze operacje na miesiącach. Kolumna kwot „amount” również zostanie przekształcona na wartości typu „float”, w celu dokładniejszych obliczeń.

Dla każdego wiersza analizowany jest jego typ. Jeśli tran_type ma wartość „income”, transakcja klasyfikowana jest jako przychód. Jeżeli tran_type znajduje się na liście kodów wydatków (EXP_ALIAS, np. „expense”, „bill_payment”, „liab_payment”, „goal_deposit”), zostaje zakwalifikowany jako wydatek. Wszystkie pozostałe transakcje, nieistotne z punktu widzenia prognozy (np. pominięte typy lub nieoczekiwane wartości), otrzymują kategorię „ignore” i są usuwane z dalszej analizy. Nowa kolumna „month” zawiera datę odpowiadającą pierwszemu dniu danego miesiąca, co pozwala na grupowanie transakcji miesięcznych. Jest tak, ponieważ df["date"].dt.to_period("M") zmienia daty na format „rok.miesiąc”, a następnie „.dt.to_timestamp()” zmienia je z powrotem na dokładną datę odpowiadającą 1 dniowi miesiąca. Następnie dane są grupowane według miesiąca i kategorii „groupby(["month", "cat"])”, a sumy kwot transakcji dla każdej pary miesiąca i kategorii są wyliczane za pomocą „sum()”.Wynikowe dane są łączne w tabelę „unstack(fill_value=0)”, w której każdy wiersz reprezentuje dany miesiąc, a kolumny sumaryczną wartość przychodów i wydatków w tym miesiącu. Sortowanie według indeksu „sort_index()” zapewnia chronologiczny porządek miesięcy.

Kod. 7. Winsoryzacja dancyh

```
1     def _winsorize(series, q=OUTLIER_Q):
2         s = series.astype(float)
3         lo, hi = s.quantile(q), s.quantile(1 - q)
4         return s.clip(lo, hi)
5
6     inc_train = _winsorize(monthly["income"])
7     exp_train = _winsorize(monthly["expense"])
```

Źródło: Opracowanie własne.

Przed zastosowaniem modeli predykcyjnych, wartości miesięczne podlegają procesowi winsoryzacji z progiem 5%. Ma to na celu ograniczenie wpływu wartości ekstremalnych na finalny model prognozy. W rezultacie uzyskujemy dane stabilniejsze i mniej podatne na zniekształcenia pojedynczymi nietypowymi wartościami, co poprawia jakość prognozy (Kod. 7).

Funkcja „_winsorize” przyjmuje jako argument szereg liczbowy oraz parametr kwantyla obcięcia q (domyślnie ustawiony na 0,05, czyli 5%). Na początku wszystkie wartości są przekształcane do typu „float”, a następnie wyznaczane są dolny „lo” i górny „hi” próg, odpowiadające wartościom odpowiednich kwantyli, czyli wartości, poniżej i powyżej których znajdzie się odpowiednio 5% najmniejszych i największych obserwacji. Funkcja „clip(lo, hi)” zastępuje wszystkie wartości mniejsze niż dolny próg przez sam próg dolny oraz wartości większe niż próg górny przez próg górny.

W efekcie wywołania funkcji dla „inc_train” i „exp_train” otrzymujemy dwa szeregi czasowe, w których wartości ekstremalne zostały przycięte do ustalonych progów. Dzięki temu dane przekazywane do modelu są bardziej odporne na zniekształcenia, a sama prognoza bardziej wiarygodna.

Kod. 8. Modelowanie szeregu czasowaego metodą Exponential Smoothing

```
inc_model = ExponentialSmoothing(
    inc_train, trend="add", seasonal=None,
    initialization_method="estimated").fit()

exp_model = ExponentialSmoothing(
    exp_train, trend="add", seasonal=None,
    initialization_method="estimated").fit()
```

Źródło: Opracowanie własne.

Głównym modelem wykorzystanym w predykcji jest wygładzanie wykładnicze z addytywnym trendem, zaimplementowane z pomocą biblioteki statsmodels . Brak komponentu sezonowego podyktowany jest niewystarczającą ilością danych historycznych. Wygładzanie wykładnicze z trendem addytywnym ujmuje trend wzrostowy lub malejący w danych historycznych (Kod. 8).

Każdy szereg czasowy jest modelowany osobno jako przychód i wydatek. Dla szeregów „inc_train” i „exp_train” tworzony jest model addytywny z wykorzystaniem argumentu „trend=add”. Ustawienie „seasonal=None” oznacza, że nie patrzymy na sezonowość, a parametr „initialization_method="estimated" pozwala algorytmowi automatycznie dobrać optymalne wartości początkowe parametrów modelu na podstawie dostępnego szeregu. Metoda „fit()” uruchamia proces dopasowania modelu uczenia do przekazanych danych historycznych i wyznacza parametry opisujące obserwowany przebieg szeregów czasowych.

Kod. 9. Generowanie prognozy na przyszłe miesiące

```
future_idx = pd.date_range(
    start=monthly.index.max() + pd.offsets.MonthBegin(),
    periods=months, freq="MS"
)

inc_forecast = inc_model.forecast(months).tolist()
exp_forecast = exp_model.forecast(months).tolist()
```

Źródło: Opracowanie własne.

Po dopasowaniu modeli, generowane są prognozy na kolejne 6 miesięcy. Indeksy czasowe kolejnych miesięcy tworzone są za pomocą biblioteki pandas. Finalne prognozy konwertowane są do wygodnej struktury, czyli listy słowników, zawierającej miesiąc oraz przewidywane wartości przychodów i wydatków, zaokrąglone do dwóch miejsc po przecinku (Kod. 9).

Funkcja „pd.date_range” tworzy listę kolejnych pierwszych dni miesięcy. Parametr „pd.offsets.MonthBegin()” ustala datę początkową zakresu prognozowanych miesięcy, a „monthly.index.max()” zwraca ostatni miesiąc dostępny w historycznych danych.

Następnie „inc_forecast” i „exp_forecast” przechowują liczby wskazujące, jakie przychody i wydatki przewiduje model dla każdego z nadchodzących miesięcy. Użycie metody „forecast(months)” automatycznie oblicza wartości dla kolejne miesiące, a wynik jest zamieniany na prostą listę liczb za pomocą „tolist()”.

Kod. 10. Generowanie prognozy na przyszłe miesiące

```
1      forecast_out = []
2      for idx, inc, exp in zip(future_idx, inc_forecast,
3            exp_forecast):
4                forecast_out.append({
5                    "month": idx.strftime("%Y-%m"),
6                    "income": round(float(inc), 2),
7                    "expense": round(float(exp), 2)})
8
9      history_out = []
10     for idx, row in monthly.iterrows():
11           history_out.append({
12               "month": idx.strftime("%Y-%m"),
13               "income": round(float(row.get("income", 0.0)), 2),
14               "expense": round(float(row.get("expense", 0.0)), 2)
15                })
```

Źródło: Opracowanie własne.

Otrzymane prognozy oraz dane historyczne są następnie konwertowane na wygodną strukturę słownikową, gotową do zwrócenia przez endpoint API i prezentacje użytkownikowi (Kod. 10).

Dla każdego indeksu z future_idx oraz odpowiadających mu prognozowanych wartości inc_forecast i exp_forecast tworzony jest słownik z kluczami: "month" jako miesiąc w formacie „YYYY-MM”, "income" jako prognozowana wartość przychodów i "expense" jako prognozowana wartość wydatków.

Analogicznie, dla każdego indeksu "idx" i wiersza "row" w "monthly.iterrows()" budowany jest słownik zawierający: "month", "income" jako suma rzeczywistych przychodów i "expense" jako suma rzeczywistych wydatków.

Rozdział IV. Testy rozwiązania

Proces testowania funkcjonalności aplikacji przeprowadzono ręcznie, zgodnie z wcześniej przygotowanymi scenariuszami testowymi. Głównym celem testów było upewnienie się, że wszystkie założone funkcjonalności działają poprawnie i zgodnie z oczekiwaniami. Testy obejmują każdą z wdrożonych funkcjonalności:

- proces rejestracji i logowania użytkowników, w tym poprawność haszowania haseł oraz zabezpieczenia dostępu,
- dodawanie, edytowanie oraz usuwanie zadań i wydarzeń w kalendarzu, w tym sprawdzanie poprawności zapisywanych danych
- zarządzanie finansami: dodawanie przychodów, wydatków, przelewów między kontami oraz kontrola ich wpływu na saldo kont użytkownika
- zarządzanie zobowiązaniami finansowymi i celami oszczędnościowymi, wraz z kontrolą poprawności wyliczeń kwot rat i odsetek
- funkcjonalność prognozowania finansowego i poprawność działania algorytmów predykcyjnych
- mechanizmy zabezpieczeń.

W trakcie testów ręcznych stosowano podejście oparte na analizie rezultatów każdej operacji zarówno w interfejsie użytkownika, jak i bezpośrednio w bazie danych. Po wykonaniu każdego przypadku testowego sprawdzano, czy dane zostały poprawnie zapisane, zaktualizowane lub usunięte w bazie danych aplikacji.

Dodatkowo została oceniona skuteczność modelu predykcji finansowej na podstawie przeprowadzonych testów dla użytkownika, który posiada 12 miesięcy danych finansowych. Oprócz oceny modelu poprzez porównanie danych prognozowanych z danymi historycznymi, zostały obliczone klasyczne metryki regresji [18]. Do oceny zostały wzięte:

- MAE (Mean Absolute Error) czyli średni błąd bezwzględny między wartością rzeczywistą a prognozowaną, mówi nam, o ile średnio się pomyliliśmy w prognozie.
- RMSE (Root Mean Suared Error) pierwiastek z średniego kwadratu błędu, jest wrażliwszy na duże odchylenia.
- MAPE (Mean Absolute Percentage Error) – średni procentowy błąd predykcji, pozwala on ocenić względną dokładność modelu.

```
>>> METRYKI MODELU PRZYCHODOW:
  MAE: 226.25
  RMSE: 321.69
  MAPE: 3.02
>>> METRYKI MODELU WYDATKOW:
  MAE: 510.31
  RMSE: 667.7
  MAPE: 7.75
```

Rys. 37. Wyniki obliczeń miar dokładności

Źródło: Opracowanie własne.

Na rysunku, przedstawiono wyniki obliczeń wykonanych na danych przykładowego użytkownika (Rys. 37). Dla przychodów błąd względny MAPE wynosi zaledwie 3.02%, co oznacza bardzo dobre dopasowanie modelu do danych użytkownika. Taki wynik świadczy o tym, że zmienność przychodów w czasie jest stosunkowo niska i dobrze poddaje się predykcji przy użyciu modelu wygładzania wykładniczego z trendem addytywnym.

W przypadku wydatków MAPE wynosi 7.75%, co również uznaje się za wynik bardzo dobry. Wartość RMSE na poziomie 667 zł może wynikać z sezonowych skoków w wydatkach (np. opłaty roczne, święta), których model wygładzający nie zawsze odwzorowuje idealnie, zwłaszcza bez komponentu sezonowości.

```
>>> MIESIECZNE SUMY HISTORYCZNE agregacja z ORM-u
  2024-05:  income =    8580.00   expense =    6731.33
  2024-06:  income =    7020.00   expense =    6029.76
  2024-07:  income =    6900.00   expense =    5953.77
  2024-08:  income =    7100.00   expense =    5977.04
  2024-09:  income =    7640.00   expense =    8129.16
  2024-10:  income =    7340.00   expense =    5065.84
  2024-11:  income =    7820.00   expense =    6476.17
  2024-12:  income =    7340.00   expense =    7597.91
  2025-01:  income =    7400.00   expense =    5856.41
  2025-02:  income =    7400.00   expense =    6556.80
  2025-03:  income =    7400.00   expense =    6162.91
  2025-04:  income =    7400.00   expense =    6535.67
>>> PROGNOZA FINANSOWA (kolejne miesiace):
  2025-05: income = 7395.13 , expense = 6539.32
  2025-06: income = 7391.93 , expense = 6557.83
  2025-07: income = 7388.73 , expense = 6576.33
  2025-08: income = 7385.53 , expense = 6594.84
  2025-09: income = 7382.33 , expense = 6613.34
  2025-10: income = 7379.13 , expense = 6631.85
```

Rys. 38. Wyniki przewidywań przykładowych danych

Źródło: Opracowanie własne.

Na podstawie wyników (Rys. 38) można stwierdzić, że model działa zgodnie z założeniami. Zarówno prognozowane przychody, jak i wydatki na najbliższe miesiące są stabilne i odzwierciedlają tendencje widoczne w danych historycznych. Model nie przewiduje żadnych nagłych skoków ani spadków, a prognozy są zbliżone do wartości obserwowanych w poprzednich miesiącach. Taki efekt świadczy o tym, że wybrana metoda dobrze sprawdza się przy szeregach czasowych o niewielkiej liczbie obserwacji i umiarkowanej zmienności.

Testy przeprowadzono również pod kątem kompatybilności z popularnymi przeglądarkami internetowymi, takimi jak Google Chrome, Mozilla Firefox czy Opera. Dzięki temu upewniono się, że aplikacja działa poprawnie i prezentuje spójny interfejs użytkownika niezależnie od używanej platformy.

Przeprowadzone testy pozwoliły na wykrycie i poprawę drobnych błędów funkcjonalnych oraz wizualnych, takich jak niepoprawne wyświetlanie danych, czy błędy aktualizacji salda kont. Dodatkowe testy prognozowania danych i analiza wyników pokazują, że zastosowany model predykcji działa skutecznie i trafnie odwzorowuje rzeczywiste dane finansowe użytkownika. Otrzymane wyniki stanowią solidną podstawę do dalszego rozwoju aplikacji, w kierunku uwzględnienia takich elementów jak sezonowość (np. świąteczne wydatki, opłaty roczne) czy nieregularne zdarzenia finansowe, co dodatkowo zwiększy precyzję prognoz i wartość praktyczną systemu.

Ostatecznie aplikacja przeszła pełen cykl testów, które potwierdziły poprawność zaimplementowanych funkcjonalności oraz stabilność działania systemu.

Podsumowanie

Przedmiotem niniejszej pracy było zaprojektowanie i implementacja aplikacji umożliwiającej integrację zarządzania zadaniami oraz finansami osobistymi. Główny cel pracy, czyli stworzenie elastycznego i przyjaznego użytkownikowi narzędzia do kompleksowego zarządzania czasem i budżetem, został zrealizowany.

Prace nad aplikacją rozpoczęto od analizy potrzeb użytkownika oraz przeglądu podobnych narzędzi dostępnych na rynku, co pozwoliło określić zestaw kluczowych funkcjonalności oraz wymagań niefunkcjonalnych. W kolejnych etapach zaprojektowano strukturę bazy danych, architekturę aplikacji oraz interfejsy użytkownika. Całość rozwiązania oparta została na technologiach Django (Python) po stronie backendu oraz JavaScript i Chart.js po stronie frontendowej. Wykorzystano także narzędzia statystyczne do predykcji przychodów i wydatków.

Aplikacja została przetestowana ręcznie, przechodząc przez różnorodne scenariusze użytkowania, testując pod kątem poprawności zapisu danych i spójności między modułami, jak i stabilności działania w różnych przeglądarkach. Przeprowadzono również testy modelu predykcji uzyskując satysfakcjonujące wyniki.

Wśród mocnych stron aplikacji należy wymienić intuicyjny interfejs oraz dynamiczne generowanie danych i wykresów w czasie rzeczywistym. System pozwala użytkownikowi nie tylko zarządzać codziennymi zadaniami i budżetem, ale także lepiej zrozumieć swoje nawyki oraz planować przyszłość finansową. Wbudowane wykresy analityczne wspomagają podejmowanie decyzji.

W przyszłości planowany jest dalszy rozwój aplikacji, przede wszystkim poprzez rozbudowę modułu analizy finansowej o bardziej zaawansowane modele predykcyjne, personalizację alertów oraz automatyzację procesów budżetowych. Szczególny nacisk zostanie położony na uwzględnienie sezonowości w analizie wydatków, co pozwoli dokładniej przewidywać okresowe wzrosty kosztów. Wszystkie te zmiany mają na celu jeszcze lepsze dopasowanie rozwiązania do realnych potrzeb użytkowników.

Bibliografia

[1] https://media.big.pl/informacje-prasowe/844005/kosztowny-stres-jak-z-problemami-finansowymi-radza-sobie-polacy [dostęp: 2025.05.12]
[2] https://web.budgetbakers.com
[3] https://app.spendee.com/auth/login
[4] https://app.todoist.com/auth/login
[5] https://www.cs.unh.edu/~cs619/slides/usecase.pdf [dostęp: 2025.05.05]
[6] Julia Plekhanova, Evaluating web development frameworks: Django, Ruby on Rails and CakePHP, Temple University. Dostępne na: https://ibit.temple.edu/wpcontent/uploads/2011/03/IBITWebframeworks.pdf [dostęp: 2025.05.25]
[7] Quan Nguyen, Hands-On Application Development with PyCharm, Packt Publishing, UK, 2019
[8] Lutz M., Wprowadzenie Python wydanie IV, Helion, Polska, 2011
[9] https://www.djangoproject.com/foundation/
[10] https://www.jetbrains.com/help/datagrip/postgresql.html#kerberos_authentication
[11] https://www.statsmodels.org/stable/index.html
[12] https://pypi.org/project/psycopg2-binary/
[13] Eli Bressert, SciPy and NumPy: An Overview for Developers, O'Reilly, USA, 2012
[14] https://www.chartjs.org/docs/latest/
[15] https://businessinsider.com.pl/poradnik-finansowy/kredyty/czym-sa-raty-annuitetowe/zh6mcc1
[16] https://facebook.github.io/prophet/docs/quick_start.html#data-requirements
[17] https://www.statsmodels.org/stable/examples/notebooks/generated/exponential_smoothing.html
[18] Alexei Botchkarev, Performance Metrics (Error Measures) in Machine Learning Regression, Forecasting and Prognostics: Properties and Typology, University Toronto, Canada, 2019 Dostępne na : https://arxiv.org/pdf/1809.03006 [dostęp: 2025.05.25]

Spis rysunków

Rys. 1. Widok aplikacji Wallet 6
Rys. 2. Widok aplikacji Spendee 7
Rys. 3. Widok aplikacji Todoist 8
Rys. 4. Schemat komunikacji aplikacji 12
Rys. 5. Model danych część 1 15
Rys. 6. Model danych część 2 16
Rys. 7. Diagram przypadków użycia dla zadań, wydarzeń i nawyków 18
Rys. 8. Diagram przypadków użycia dla finansów 19
Rys. 9. Strona startowa aplikacji 25
Rys. 10. Strona rejestracji aplikacji 26
Rys. 11. Strona logowania aplikacji 27
Rys. 12. Kalendarz i listą zadań krótkich 28
Rys. 13. Formularz wydarzenia 28
Rys. 14. Formularz zadania krótkiego 29
Rys. 15. Kalendarz miesięczny 30
Rys. 16. Kalendarz tygodniowy 30
Rys. 17. Lista zadań długich 31
Rys. 18. Formularz zadania długoterminowego 31
Rys. 19. Harmonogram nawyków 32
Rys. 20. Wykres i statystyka dla nawyków 33
Rys. 21. Podsumowanie miesięczne wydatków 34
Rys. 22. Statystyki transakcji i lista rachunków 35
Rys. 23. Widok kont użytkownika 35
Rys. 24. Formularz wydatku 36
Rys. 25. Formularz przychodu 36
Rys. 26. Formularz przelewu środków 37
Rys. 27. Formularz opłaty cyklicznej 37
Rys. 28. Formularz dodania konta 38
Rys. 29. Formularz ustawień budżetu miesięcznego 39
Rys. 30. Lista transakcji z możliwością filtracji 39
Rys. 31. Lista celów oszczędnościowych 40
Rys. 32. Formularz celu 41

Rys. 33. Lista zobowiązań 41
Rys. 34. Formularz zobowiązania 42
Rys. 35. Historia transakcji zobowiązania 42
Rys. 36. Wykres danych historycznych i predykcji 43
Rys. 37. Wyniki obliczeń miar dokładności 56
Rys. 38. Wyniki przewidywań przykładowych danych 56

Załączniki

1. Płyta CD/DVD zawierająca:
    a. Prezentację wyników pracy dyplomowej
    b. Kody źródłowe oprogramowania
    c. Biblioteki programowe niezbędne do zbudowania i uruchomienia oprogramowania